CONTAMINATION-AWARE WASTE CLASSIFICATION SYSTEM

Thesis submitted in partial fulfilment of the requirements of the degree of

Bachelor of Technology

in

Information Technology and Data Science

by

Sumit Rathi (2022-B-23102004A)
Sanjana Arun (2022-B-11052004)
Seanna Martin (2022-B-08112004C)

Under the Supervision of

Dr. Sandeep Kulkarni

School of Engineering
AJEENKYA DY PATIL UNIVERSITY, PUNE

April 2025

Certificate

Date: April 25, 2025

This is to certify that the dissertation entitled **"Contamination-Aware Waste Classification System"** is a bonafide work of **"Sumit Rathi (2022-B-23102004A), Sanjana Arun (2022-B-11052004) & Seanna Martin (2022-B-08112004C)"** submitted to the School of Engineering, Ajeenkya DY Patil University, Pune in partial fulfilment of the requirement for the award of the degree of **"Bachelor of Technology in Information Technology and Data Science"**.

Dr. (Dean Name)
Dean

Dr. Sandeep Kulkarni
Supervisor

Internal Examiner

External Examiner

Supervisor's Certificate

Date: April 25, 2025

This is to certify that the dissertation entitled **"Contamination-Aware Waste Classification System"** submitted by **Sumit Rathi (2022-B-23102004A), Sanjana Arun (2022-B-11052004) & Seanna Martin (2022-B-08112004C)**, is a record of original work carried out by them under my supervision and guidance in partial fulfilment of the requirements of the degree **Bachelor of Technology in Information Technology and Data Science** at School of Engineering, Ajeenkya DY Patil University, Pune, Maharashtra-412105.

Neither this dissertation nor any part of it has been submitted earlier for any degree or diploma to any institute or university in India or abroad.

Dr. Sandeep Kulkarni
Supervisor

Declaration of Originality

We, **Sumit Rathi (2022-B-23102004A), Sanjana Arun (2022-B-11052004) & Seanna Martin (2022-B-08112004C)**, hereby declare that this dissertation entitled **"Contamination-Aware Waste Classification System"** presents our original work carried out as bachelor students of School of Engineering, Ajeenkya D Y Patil University, Pune, Maharashtra.

To the best of our knowledge, this dissertation contains no material previously published or written by another person, nor any material presented by us for the award of any degree or diploma of Ajeenkya DY Patil University, Pune or any other institution. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Date: 25 April, 2025
Place: Lohegaon, Pune

Sumit Rathi
Sanjana Arun
Seanna Martin

Acknowledgement

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this thesis. First and foremost, we are deeply grateful to our supervisor, **Dr. Sandeep Kulkarni**, for his invaluable guidance, continuous support, and encouragement throughout this research work. His expertise and insight have been instrumental in shaping this project.

Sumit Rathi
Sanjana Arun
Seanna Martin

Contents

List of Figures

List of Tables

Chapter 1

INTRODUCTION

The Cloud First era has revolutionized the digital economy in the world. With organizations progressing beyond mere cloud adoption to Multi-Cloud Strategic Maturity, the necessity to integrate management platforms has emerged as a serious operational necessity.

1.1 Historical Context of Computing

There have been a number of paradigms of transformation in modern computing, all of which have made the user and the underlying hardware more and more abstract. It is important to understand how these paradigms have developed to fully recognize the value proposition of the Nebula Multi-Cloud Command Center.

1.1.1 The Mainframe Era (1950s - 1970s)

Early computer technology saw the use of massive computers which occupied whole rooms. These systems were centralized and users accessed these systems through dumb terminals. Though very powerful at the time, they were prohibitively priced, difficult to scale, and were a single point of failure. It was highly manualized management that demanded a specialized workforce.

1.1.2 Client-Server Architecture (1980s - 1990s)

The client-server model was the result of the emergence of the Personal Computer (PC). Calculations were performed over a server (where data or services are delivered) and a client (where they are requested). Although more flexible compared to the mainframes, this period brought about the so-called silo issue where various departments operated with their own physical servers, resulting in inefficient use of the hardware resources and excessive maintenance expenses.

1.1.3 The Virtualization Revolution (2000s)

The relationship between software and hardware was changed by virtualization. Hyper- visors enabled more than one Virtual Machines (VMs) to operate on a physical server. It was the technological antecedent to cloud computing and allowed efficient resource utilization and the ability to spin up servers in minutes as opposed to weeks.

1.2 The Rise of Cloud Computing

Cloud computing is the natural extension of virtualization, wide network access and automated management. According to the National Institute of Standards and Technology (NIST) [1], cloud computing refers to a model to facilitate ubiquitous, convenient, on-demand network access to a common pool of configurable computing resources.

1.2.1 NIST Service Models

To get the picture of what Nebula platform covers, we need to classify the services that it deals with:

1. **Infrastructure as Service (IaaS)**: Offers basic computing, including Virtual Machines (AWS EC2, Azure VM), storage and networks. This layer is heavily integrated by Nebula.

2. **Platform as Service (PaaS)**: It is a platform that enables customers to create, execute, and manage applications without having the complexity of infrastructure. Examples are AWS Lambda and Azure Functions, which Nebula is managing as Network Functions.

3. **Software as Service (SaaS)**: Software is sold on a subscription model and is centrally hosted. Although Nebula is a SaaS-like platform, it controls the underlying IaaS and PaaS to facilitate enterprise SaaS deployments.

1.2.2 NIST Deployment Models

- **Public Cloud**: It is owned and managed by a third party service provider (AWS, Azure, GCP).

- **Private Cloud**: Resources known to only one business or organization.

- **Hybrid Cloud**: A combination of two or more clouds (community, private, or public) that are distinct entities, yet are interconnected.

- **Multi-Cloud**: This is where a number of public clouds of various providers are used. Nebula is actually created to conquer this space.

1.3 The Multi-Cloud Strategic Shift

A multi-cloud strategy is another approach that organizations are employing to prevent Vendor Lock- in. Nonetheless, this transition presents important managerial strife.

1.3.1 Drivers for Multi-Cloud Adoption

- **Best-of-Breed Services**: Data analytics with GCP and retaining enter- prise apps on Azure.
- **Data Sovereignty**:Storing data in particular physical locations to abide by GDPR or local laws.
- **Negotiation Leverage**: Presence in several clouds eliminates dependency on one vendor and their pricing model.

1.4 Problem Statement and Motivation

Multi-cloud management is disjointed, despite its advantages. At this time, an administrator should:

1. Control high-security credentials in three portals.
2. Get familiar with different naming (e.g., Security Groups in AWS and NSG in Azure).
3. Manage deployments that are long-running and frequently crash in the browser.

Nebula is motivated by the need to develop a Unified Control Plane which encapsulates these complexities into one, high-performance API and User Interface.

1.5 Project Objectives and Scope

The particular targets of this project are:

- Achieve 100
- Minimize troubleshooting with LLM log analysis.
- Automatize resource provisioning processes.
- A secure credential vault by way of application-layer encryption.
- Stream the deployment logs in real time with WebSockets or polling.

The area includes the early work on the core engine, security vault and a React-based interactive cockpit.

Chapter 2

LITERATURE SURVEY

Over the last 2 decades, there has been a significant growth in the cloud management industry. In order to gain insight into where Nebula fits into the existing marketplace, we carried out a comprehensive survey of three different types of tools Public Cloud Native Orchestrator, Third-Party IaC Tools, and Emerging Unified Platforms.

2.1 Critical Analysis of 10+ Cloud Management Tools

2.1.1 Cloud-Native Orchestrators (Provider-Specific)

1. **AWS CloudFormation**: CloudFormation is a powerful, JSON/YAML based service which enables declarative definition of infrastructure in AWS. The key benefit of it is that it is highly integrated with AWS IAM and introduction of new services. But the biggest disadvantage is its walled garden characteristic, as it cannot deal with resources in Azure or GCP.
2. **Azure Resource Manager (ARM)**: Azure templates are used to define infrastructure using ARM templates. Though it provides very powerful ”Blueprints” to compliance, its syntax is also notoriously verbose and difficult to debug, when compared to Terraform.
3. **GCP Deployment Manager**: Google Cloud native IaC service. It supports Python and Jinja2 templates, providing more programmability than CloudFormation but with no huge support of third-party tools.

2.1.2 Cross-Cloud Infrastructure as Code (IaC) Tools

1. **Terraform (HashiCorp)**: Terraform is the standard in the industry and the heart of Nebula engine. Its Provider architecture enables it to communicate to any API. Non-DevOps engineers, however, tend to be scared away by its CLI-only status and complexity of State Management.
2. **Pulumi**: A relatively new tool, a competitor to Terraform, which uses standard pro- gramming languages (Python, Go, Node.js). Although it is flexible, its ecosystem of predefined

modules is small in comparison to Terraform.
3. **Ansible**: Is mainly a configuration management tool, but often used for provisioning. It is less appropriate to the Infrastructure Lifecycle management (e.g., the time when a resource has been deleted or changed) because of its imperative nature.
4. **Chef & Puppet**: Legacy configuration management tools that have been mostly replaced by containerization (Docker) and IaC (Terraform).

2.1.3 Unified Multi-Cloud Management Platforms

1. **VMware Cloud (VMC)**: Offers a uniform vSphere experience on other clouds. Its main disadvantage is that it is very expensive to license as an enterprise, and is therefore unavailable to small developers. 2. **Google Anthos**: This is a managed service with which it is possible to manage Kubernetes in both AWS and Azure. It is however, purely containerized workload-oriented and there is no general-purpose AWS or Azure VM/Storage management.
3. **Morpheus Data**: A commercial ”Cloud Management Platform” (CMP). Although potent, it is a Black Box that can be hardly extended to custom AI-based troubleshooting, and that is where Nebula offers its own value.

2.2 Summary of Comparative Analysis

2.3 Research Gaps in Published IEEE Literature

We used 5 key research streams in the IEEE Xplore online library pertaining to multi-cloud management. The following gaps were found:

2.3.1 Gap 1: The ”Log Normalization” Deficit

There is a considerable body of work (e.g., [22,28]) on Multi-Cloud Resource Dis- covery. Nonetheless, the method of normalization of error logs across clouds is not standardized.

2.3.2 Gap 2: Developer Experience (DevEx) in IaC

Recent studies on IaC (e.g., [5, 26]) are based on the ”Language” level. Research on how to make IaC accessible to non-experts by using high-level UI abstractions and background execution is lacking.

2.3.3 Gap 3: Automated Secret Injection Security

In the majority of articles on multi-cloud security (e.g., [27]) a hardware security module (HSM) is assumed. The research work by Nebula investigates on the application-layer sym-

metric encryption (AES-128) as a tool to securely maintain credential management on portable devices without the need to rely on costly hardware.

Table 2.1: Comparison of Cloud Management Tools with Nebula

Feature	**Native Tools**	**Terraform**	**Anthos**	**Nebula**
Multi-Cloud Support	No	Yes	Partial	**Full**
Unified GUI	No	No	Yes	**Yes**
AI Error Remediation	No	No	No	**Yes**
Asynchronous Tasks	No	No	Yes	**Yes**
Native API Security	Yes	Manual	Yes	**Yes**

Chapter 3

SYSTEM ANALYSIS

Any multi-cloud project will be successful or not based on the quality of the Prob- lem Space analysis before moving to the Solution Space. This chapter outlines the technical requirements, limitations and the strategic planning of the Nebula platform.

3.1 Software Requirements Specification (SRS)

We divided our requirements in terms of IEEE-830 standard [29] into non-functional and functional goals.

3.1.1 Actor Definitions

1. **Cloud Administrator**: Adds cloud credentials and administers access to the organization.
2. **DevOps Engineer**: The primary user that provisions, administers and oversees resources in clouds.
3. **System Auditor**: Checks the cost reports and inventory history to check compliance.

3.1.2 Use Case Descriptions

The interactions between actors and the system to achieve special busi- ness objectives are detailed use cases.

- **Use Case 1: Secure Credential Registration**
    1. Registering Secure Credential. User clicks Register Provider, and enters an Access Key and Secret Key.
    2. The format is validated by the Pydantic schemas [30] by the system. [**?**].
    3. Key encryption with AES-128 and storage in PostgreSQL are done in the backend.
    4. The user is redirected to the “Settings” dashboard.

- **Use Case 2: Multi-Cloud VM Provisioning**

  1. User clicks ”New VM” and selects ”AWS” as the provider.
  2. System obtains the pre-defined EC2 Terraform module.
  3. User is prompted to input instance name, size and region.
  4. System triggers a Celery task and gives a real-time status report.

3.2 Requirement Traceability Matrix (RTM)

The RTM is a document which maps and tracks user requirement with test cases. It also documents all the requirements suggested by the client and traceability of requirements in one document, at the end of the Software development life cycle.

3.3 Detailed Project Schedule (16-Week Plan)

The project was executed following a structured timeline, ensuring that every phase of the SThe project was conducted in a logical sequential manner and each SDLC stage was accorded sufficient time to conduct research, design, implementation and testing.

3.3.1 Phase 1: Research and Feasibility (Weeks 1-3)

- Multi-cloud gaps identification. - Survey of 10 or more cloud tools (as discussed in Chapter 2). - Terraform as the orchestration engine validation.

3.3.2 Phase 2: System Architecture Design (Weeks 4-6)

- High-Level Design (HLD) formalization. - Choice of tech stack: FastAPI, React, PostgreSQL, Redis and Celery. - Database schema design and entity definition.

3.3.3 Phase 3: Core Module Implementation (Weeks 7-11)

- Nebula Vault credential security: Creating the Nebula Vault. - Terraform Task Runner: Creating the Terraform Task Runner. - provider-specific HCL (AWS, Azure, GCP): Building modules.

3.3.4 Phase 4: Frontend Development (Weeks 12-14)

- Dashboard-based command center. - Backend WebSock- ets-implemented real time stream logging. - Copilot UI implementation.

3.3.5 Phase 5: Testing and Deployment (Weeks 15-16)

- Non- Functional and Functional testing (read Chapter 6). - Final documentation and writing user manual.

3.4 Risk Management and Mitigation

The problem presents various threats that have already been discovered and dealt with in the analysis stage when a large number of cloud providers are involved.

3.5 Data Dictionary for Core Entities

The following table describes the characteristics of all the objects of operations in the PostgreSQL schema.

Table 3.1: Requirement Traceability Matrix

Req ID	Requirement Description	Design Ref	Test ID
FR-01	User Authentication via JWT	Sec-Arch 4.5	TC-01
FR-02	Encrypted Credential Vault	Sec-Arch 4.5	TC-02
FR-03	AWS Resource Provisioning	Engine 5.3	TC-03
FR-04	Azure Resource Provisioning	Engine 5.3	TC-04
FR-05	GCP Resource Provisioning	Engine 5.3	TC-05
FR-06	AI Log Remediation Engine	AI-Logic 8.1	TC-06
FR-07	Real-time Status Sync	Sync-Arch 4.6	TC-07
NFR-01	Encryption at Rest (AES-128)	Sec-Arch 4.5	TC-14
NFR-02	API Latency < 1s	HLD 4.1	TC-33
NFR-03	System Uptime 99.9%	Ops-Manual	TC-35

Table 3.2: Project Risk Matrix

Risk	Impact	Mitigation Strategy	Contingency
Cloud API Rate Limiting	High	Implement exponential backoff in sync tasks.	Alert administrator.
Credential Compromise	Critical	Encryption at application layer (AES-128).	Secret rotation policy.
Terraform State Corruption	High	Use local backups and isolation per resource.	State recovery tools.
AI Hallucination	Medium	Sanitized logs and human-in-the-loop validation.	Fallback to raw logs.
Provider Outage	High	Multi-cloud redundancy (Nebula survives if 1 cloud fails).	Failover to other cloud.

Table 3.3: Nebula Data Dictionary

Entity	Attribute	Type	Description
Credential	provider_type	Enum	AWS, AZURE, or GCP
Credential	access_key	String (Enc)	Encrypted primary access key
Resource	provider_id	String	Unique cloud ID (e.g., i-12345)
Resource	status	Enum	active, pending, failed, or deleted
Task	execution_log	Text	Full STDOUT/STDERR from Terraform

Chapter 4

SYSTEM DESIGN

As mentioned earlier, the system architecture for Nebula is built with high modularity, async nature, and strong security capabilities. Chapter 3 focuses on the internal workings of the platform and its components such as the overall high-level ecosystem, database design, and the security model.

4.1 System Architecture Overview

Following the standard three-tier architecture, the platform has separate tiers, Presentation, Application, and Persistence layers. All layers are isolated via the use of Docker-Compose.

4.1.1 High-Level Ecosystem (TikZ Diagram)

Refer to the following figure that shows interactions among the user browser, internal containerized network, and external cloud provider APIs.

4.2 Entity-Relationship (ER) Design

The database (PostgreSQL) is considered the ”Global State Manager” within the platform. The inventory of all three cloud providers is stored in one normalized database schema.

4.2.1 ER Diagram (TikZ)

Referring to the following diagram below, we can see the primary entities and the relationships. Contrary to a high-level ER diagram, this one is consistent with the persistence schema outlined later in the appendix. The operational tables that make sense from the platform point of view include users, cloud credentials, provisioned resources, tasks, and activity log entries.

The following ER diagram shows that the `users` table is the root ownership entity. Each user can register multiple cloud credentials, provision multiple resources, and generate multi-

[node distance=2.2cm, auto, scale=0.8, every node/.style=scale=0.8] box = [rectangle, draw, fill=blue!5, text width=7em, text centered, rounded corners, minimum height=2.5em] sub = [rectangle, draw, dashed, inner sep=10pt, fill=gray!5] line = [draw, -latex'] [box] (browser) User Browser; [above=0.2cm of browser, font=**] Client;**
[box, below=1.85cm of browser] (backend) FastAPI Backend; [box, left=0.5cm of backend] (frontend) React (Nginx); [box, right=0.5cm of backend] (lb) Reverse Proxy; [box, below=1.5cm of backend] (redis) Redis Broker; [box, left=0.5cm of redis] (db) PostgreSQL; [box, right=0.5cm of redis] (worker) Celery Worker;
[dashed] ([xshift=-0.5cm, yshift=0.5cm]frontend.north west) rectangle ([xshift=0.5cm, yshift=-0.5cm]worker.south east); at ([yshift=0.8cm]backend.north) Docker Network Cluster;
[box, below=2.5cm of redis] (azure) Azure Cloud; [box, left=0.5cm of azure] (aws) AWS Cloud; [box, right=0.5cm of azure] (gcp) GCP Cloud;
[below=0.2cm of azure, font=] Multi-Cloud APIs;
[line] (browser) – node[left] REST (backend); [line] (browser) – (frontend);
[line] (backend) – (db); [line] (backend) – (redis); [line] (worker) – (redis); [line] (worker) -| (db);
[line] (worker) – (aws); [line] (worker) – (azure); [line] (worker) – (gcp);

Figure 4.1: Nebula: Unified Multi-Cloud Infrastructure Ecosystem

ple activity log entries. Each provisioned resource can in turn generate multiple background task records as the worker system performs create, sync, update, or delete operations. The dashed association between `cloud_credentials` and `resources` indicates an operational dependency: resources are provisioned by using a credential record, even though that linkage may be enforced at the application layer rather than through a direct foreign key column in the simplified schema.

4.3 Network Architecture and Cloud Connectivity

Resources provisioning to the cloud needs not only an API key but also the existence of a well-defined network structure. VPC Peering and Security Groups are automated by Nebula.

4.3.1 Virtual Private Cloud (VPC) Design

For every project launched in Nebula, a dedicated VPC is created. This ensures:

- **Isolation**: Resources in a project can't communicate to another project's resources by default.
- **Secure Subnetting**: Public subnets for Load Balancers and private subnets for Database/-Worker nodes.
- **Internet Gateway (IGW)**:If required for resources.

[node distance=1.6cm, auto, scale=0.58, every node/.style=scale=0.78] entity = [rectangle, draw, rounded corners, fill=green!8, inner sep=5pt] rel = [draw, thick]

[entity] (users) **USERS** **PK** id username email hashed_password created_at ;

[entity, right=3.0cm of users] (creds) **CLOUD_CREDENTIALS** **PK** id **FK** user_id provider_type access_key_enc secret_key_enc tenant_id_enc created_at ;

[entity, below=2.4cm of users] (resources) **RESOURCES** **PK** id **FK** user_id provider resource_type status provider_id metadata created_at ;

[entity, right=3.0cm of resources] (tasks) **TASKS** **PK** id **FK** resource_id task_uuid status execution_log started_at completed_at created_at ;

[entity, below=2.5cm of resources] (activity) **ACTIVITY_LOGS** **PK** id **FK** user_id action provider target_id details created_at ;

[rel] (users) – node[above] 1:N (creds); [rel] (users) – node[left] 1:N (resources); [rel] (users.south) |- node[left, pos=0.25] 1:N (activity.west); [rel] (resources) – node[above] 1:N (tasks); [rel, dashed] (creds.south) |- node[right, pos=0.28] used for provisioning (resources.north);

Figure 4.2: Nebula Database Entity Relationship Diagram Aligned with the Implemented Schema

4.4 Security Architecture Details

Nebula adopts the approach of defense in depth for protecting user data and access to cloud APIs.

4.4.1 Identity Layer (JWT Flow)

User authentication is handled via JSON Web Tokens (JWT). When a user logs in: 1. The back end validates credentials against PostgreSQL. 2. The browser receives an authorized JWT (using RS256). 3. All subsequent API calls need this token as Authorization: Bearer <TOKEN>. `Authorization: Bearer <TOKEN>`.

4.4.2 Credential Security (AES-128 Fernet)

Cloud secrets (AWS Secret Keys) are never stored in plain text. We use the Fernet implementation which provides symmetric encryption with:

- **Confidentiality**: Messages cannot be read without the key.
- **Integrity**: Messages cannot be altered without detection.
- **Freshness**: Messages cannot be replayed without detection.

[node distance=5.7cm, auto] actor = [rectangle, draw, fill=orange!10, minimum width=3em, minimum height=3em] line = [draw, -latex]
[actor] (user) User Browser; [actor, right of=user] (api) FastAPI; [actor, right of=api] (db) PostgreSQL;
[line] (user) – node POST credentials (api); [line] (api) – node Encrypt(Key) (db); [line, dashed] (db) – node Ciphertext (api);

Figure 4.3: Credential Encryption Workflow

Chapter 5

IMPLEMENTATION

The implementation layer is where the ”Nebula Engine” resides. This chapter pro- vides a deep-dive into the ”Code and Workflows” that power the Nebula Multi-Cloud platform.

5.1 Implementation Design Principles

1. **Separation of Concerns**: The API, worker and the user interface are separate services.
2. **Asynchronous Execution**: All costly ”Cloud Actions” are offloaded to workers to maintain responsiveness..
3. **Explicit State Transitions**: Each resource is either in the pending state or the active state or the failed state.
4. **Security-by-Default**: Credentials remain encrypted at rest and are only materialized in memory during task execution.

5.2 Core Implementation Workflows (TikZ Diagrams)

5.2.1 Authentication Flow (TikZ)

This sequence ensures secure, JWT-based access for every user request.

[node distance=4.5cm, auto] actor = [rectangle, draw, fill=blue!10, minimum width=3em, minimum height=3em] line = [draw, -latex]
[actor] (user) User; [actor, right of=user] (front) Frontend; [actor, right of=front] (back) Backend; [actor, right of=back] (db) PostgreSQL;
[line] (user) – node Login (front); [line] (front) – node POST /auth (back); [line] (back) – node Verify (db); [line, dashed] (back) – node JWT Result (front);

Figure 5.1: Nebula Secure Authentication Sequence

5.3 Line-by-Line Code Walkthrough: Backend Provisioning

This section provides an exhaustive explanation of the primary logic that executes Terraform and manages cloud state.

5.3.1 Task Execution: terraform_tasks.py

This module is the backbone of the background worker cluster. It uses Celery to execute Terraform without blocking the main API thread.

Listing 5.1: Provisioning Task Walkthrough

```
@celery.task(bind=True)
def provision_resource(self, resource_id: int):
    # Line 1: Fetch the resource from the database
    resource = db.query(Resource).filter(Resource.id == resource_id).first
        ()

    # Line 2-3: Decryption of Cloud Credentials
    aws_keys = vault.decrypt(resource.credential.secret_key)

    # Line 4-5: Temporary Workspace Creation
    workspace_path = f"/tmp/terraform/{resource_id}"
    os.makedirs(workspace_path)

    # Line 6-8: HCL Template Generation
    generate_hcl(workspace_path, resource.type, resource.variables)

    # Line 9-11: Process Execution (Terraform Init/Apply)
    result = subprocess.run(["terraform", "apply", "-auto-approve"],
                            cwd=workspace_path, capture_output=True, env=
                                aws_keys)

    if result.returncode == 0:
        resource.status = "active"
    else:
        resource.status = "failed"
        analyze_with_ai_copilot(result.stderr)
```

5.4 Line-by-Line Code Walkthrough: Frontend React UI

This section provides a core analysis of how the React dashboard communicates with the FastAPI backend.

5.4.1 Resource Provisioning Form: ResourceForm.tsx

This component captures user intent and dispatches it to the API.

Listing 5.2: React Resource Provisioning Hook

```
const useProvisionResource = () => {
    const [loading, setLoading] = useState(false);

    // Line 1: The provision function handles the API call
    const provision = async (payload: ResourceCreate) => {
        setLoading(true);
        try {
            // Line 2: Dispatch the request to the FastAPI endpoint
            const response = await axios.post("/api/resources/", payload);

            // Line 3: Handle the asynchronous response (Task ID)
            const taskId = response.data.task_id;

            // Line 4: Trigger the real-time log polling
            startPolling(taskId);

            toast.success("Deployment␣started␣successfully!");
        } catch (error) {
            toast.error("Failed␣to␣initiate␣deployment.");
        } finally {
            setLoading(false);
        }
    };

    return { provision, loading };
};
```

5.4.2 Main Cockpit Dashboard: Cockpit.tsx

This file serves as the primary ”Command Center,” coordinating various sub-components.

Listing 5.3: Dashboard State Management

```
const Cockpit = () => {
    // Line 1: Global State for Resources
    const [resources, setResources] = useState<Resource[]>([]);

    // Line 2: Fetching resources on mount
    useEffect(() => {
        const fetchAll = async () => {
            const data = await axios.get("/api/resources/");
            setResources(data.data);
```

```
        };
        fetchAll();
    }, []);

    // Line 3: Filtering logic for Multi-Cloud search
    const filtered = resources.filter(r =>
        r.name.toLowerCase().includes(searchTerm.toLowerCase())
    );

    return (
        <div className="flex␣bg-gray-900␣min-h-screen">
            <Sidebar />
            <main className="flex-1␣p-8">
                <Header title="Unified␣Infrastructure␣Cockpit" />
                <StatsGrid resources={resources} />
                <ResourceTable resources={filtered} />
            </main>
        </div>
    );
};
```

5.5 State Management Logic and Lifecycle

The Nebula project utilizes both React Context as well as its own hooks for the management of infrastructure state on different cloud platforms.

- **Context Provisioning**: The `CloudProviderContext` stores the registration status of AWS, Azure, and GCP accounts globally.
- **Effect Lifecycles**: If the state of the particular resource changes (for example, from 'pending' to 'active'), an effect is produced to update the local inventory without reloading the whole page.
- **Memoization**: To keep the inventory lists with many rows performant, their memoization is implemented through.

Chapter 6

RESULTS AND DISCUSSION

Validation of Orchestration Determinism and Security Vault were the goals of our testing phase.We performed performance benchmarking and functionality testing of the ”asynchronous” design.

6.1 System Performance Benchmarking

Performance benchmarking was done on the performance of our ”asynchronous” design on different clouds.Performance benchmarking was done on the performance of our ”asynchronous” design on different clouds.

6.1.1 Cloud Synchronization Latency (TikZ Chart)

Testing of time taken for retrieval and normalization of infrastructure data from AWS, Azure, and GCP.

[ybar, enlargelimits=0.15, legend style=at=(0.5,-0.15), anchor=north,legend columns=-1, ylabel=Time (seconds), symbolic x coords=AWS,Azure,GCP,S3/GCS, xtick=data, nodes near coords, nodes near coords align=vertical,] coordinates (AWS,1.2) (Azure,1.8) (GCP,1.1) (S3,0.8); Avg Sync Latency

Figure 6.1: Avg Provisioning Synchronization Latency across Cloud Providers

These results confirm that our FastAPI background tasks are highly efficient, maintaining a ”Unified View” of resources with minimal overhead.

6.2 Testing Strategy and Results

The Nebula has been tested thoroughly using the ”Shift-Left” method, whereby the unit test was performed iteratively throughout the development cycle.

6.2.1 Massive Functional Test Execution Summary (50+ Items)

The following table summarizes 50 critical test cases covering the entire platform.

6.3 Failure Analysis Case Studies (AI Engine)

One of the important things to note about Nebula is the accuracy of the failure analysis engine. Here are three examples.

6.3.1 Case Study 1: Missing IAM Permissions

While trying to deploy the S3 bucket within the constrained region using Terraform, it was shown as "Access Denied." The AI Copilot checked the logs and correctly figured out that the IAM user didn't have the required S3:CreateBucket permission in the particular VPC.

6.3.2 Case Study 2: Provider Quota Limits

In another case, while testing the bulk provisioning (TC-12), Azure showed "Quota Exceeded." The AI successfully understood the problem and suggested increasing the quota in the Azure portal.

Table 6.1: Nebula Core Test Execution Summary (Exhaustive Matrix)

ID	Module	Test Scenario	Result
TC-01	Authentication	Login with JWT and RSA Token	PASS
TC-02	Security	Encrypted Credential Vault	PASS
TC-03	Security	AWS Secret Key Encryption	PASS
TC-04	Provisioning	AWS S3 Bucket Creation	PASS
TC-05	Provisioning	AWS EC2 Instance Launch	PASS
TC-06	Provisioning	Azure Storage Account	PASS
TC-07	Provisioning	Azure VM Launch	PASS
TC-08	Provisioning	GCP GCS Bucket Creation	PASS
TC-09	Provisioning	GCP GCE Launch	PASS
TC-10	AI-Copilot	Error Analysis: 403 Access Denied	PASS
TC-11	AI-Copilot	Error Analysis: VPC Quota Exceeded	PASS
TC-12	Async Engine	Worker Load Scaling (50 Tasks/s)	PASS
TC-13	Sync Module	Inventory Sync: AWS EC2	PASS
TC-14	Sync Module	Inventory Sync: Azure VM	PASS
TC-15	Sync Module	Inventory Sync: GCP GCE	PASS
TC-16	UI/UX	Dashboard Resource Counters	PASS
TC-17	UI/UX	Real-time Log Stream (WebSockets)	PASS
TC-18	API	Credential Decryption Response Time	PASS
TC-19	API	CRUD Operation on Resource Tags	PASS
TC-20	Network	VPC Peering Verification	PASS
TC-21	Database	PostgreSQL Query Latency	PASS
TC-22	Vault	AES-128 Symmetric Cipher Check	PASS
TC-23	UI	Responsive Design (Mobile Device)	PASS
TC-24	Error	Handling 503 Backend Timeout	PASS
TC-25	Sync	Cross-Region Resource Discovery	PASS
TC-26	Security	SQL Injection Protection	PASS
TC-27	Security	XSS Protection in Dashboard	PASS
TC-28	AI	Log Scrubbing for PII/Secrets	PASS
TC-29	IaC	Terraform State Integrity Check	PASS
TC-30	UI	1-Click Repair Execution	PASS
...	**...**	**Cases TC-31 to TC-50**	**PASS**

Chapter 7

CONCLUSION

The emergence of Nebula shows that multi-cloud orchestration can become much more accessible if infrastructure automation, security, observability, and debugging are considered as a single challenge. Instead of constructing a shallow interface on top of the cloud provider's APIs, the project creates a full execution pipeline consisting of the following components: the user-facing control tower, the credential storage, the asynchronous task executor, the Terraform-driven provisioning stage, and the artificial intelligence-driven debugging component.

7.1 Summary of the Work Accomplished

The project emerged out of a concrete need. Today's firms operate within an environment where multiple vendors provide services for their operations; Amazon Web Services are chosen for coverage, Microsoft Azure for enterprise connectivity, and Google Cloud Platform for analysis and abstraction. While this helps in being resilient and having more negotiating leverage, it leads to the problem of operational fragmentation. This is why the engineers have to deal with a variety of naming conventions, authentication schemes, and logging practices.

Throughout the report, we established the conceptual basis for cloud comput- ing, reviewed available tools, highlighted the discrepancy between native provider solutions and cross-cloud processes, and ultimately translated this knowledge into a practical software architecture. The resulting framework creates a resource model that enables the user to conceptualize their actions in terms of their intentionality instead of their implementation details through various vendors. A virtual machine, storage bucket, or network perimeter becomes just another element in this normalized model instead of a tool chain for each vendor.

From an implementation standpoint, some tangible results have been reached. A FastAPI backend ensures consistency in the API for any actions performed by the user and the manipulation of resources. Celery workers handle long-running operations outside of the request-response cycle to keep the user interface snappy. Persistent state is kept using PostgreSQL

databases, whereas the queueing layer uses Redis. Terraform operates as the execution mechanism, ensuring auditability and consistency. In addition to these components, the dashboard aggregates all information about the inventory, tasks, and suggestions for troubleshooting into one cockpit.

Moreover, security was not considered an afterthought. Credential data is encrypted before being stored and decrypted within the specific execution environment where it is required. Such an approach decreases the exposure area in case of a breach of the database and creates a more pronounced distinction between the state of the application and the state of secrets. Additionally, the AI copilot was presented as an assistant rather than an independent entity. Logging is sanitized, the scope of the analysis is confined to the operation environment, and the final decision is left for the human operator.

7.2 Validation Against Project Objectives

Each of the stated objectives can be revisited, one after another, to determine how well their vision was implemented.

1. **Unified visibility across cloud providers**: The project proposes a unified inventory view for multiple cloud providers. Even though the internal name differs among them, the resource is displayed in a universal form of Provider > Type > Region > State > Metadata on the dashboard. Thus, the objective of minimizing the switch cost for the operator is fulfilled.

2. **Standardized provisioning workflows**: Nebula transforms a user command into a cloud provider-specific Terraform invocation but keeps the frontend consistent. The user invokes the infrastructure using a single portal; however, the semantic of the invocation is different from one cloud provider to another.

3. **Reduced troubleshooting time**: The AI copilot minimizes the gap between raw error logs and remediation actions. Instead of asking the operator to manually review hundreds of lines of logs generated by the provider, the system identifies the possible failure type and suggests a resolution.

4. **Secure credential handling**:The provider secrets are stored in an encrypted vault and are never materialized in plain text. They are only decrypted temporarily for a single operation to satisfy the system's security needs.

5. **Responsive user experience**: Provider secrets are kept in an encrypted vault and are never stored in plaintext form. They are decrypted temporarily only for one-time use to meet the system's security requirements.

Overall, these findings indicate that the project achieves its main academic and engineering goals. It is not an idealized mockup but an actual cloud automation framework based on practical limitations in asynchronous execution, secrets management, reconciliation, error handling, and provider differences.

7.3 Key Engineering Lessons

7.3.1 Abstraction Must Be Honest

One of the key lessons learned during the project implementation was that a good abstraction did not eliminate differences but rather managed them properly. Although the three providers had similar services, their interfaces were still different enough to make a simple layer misleading if some limitations specific to each of the providers were ignored. Consequently, a two-layer approach was beneficial for the platform as it allowed for simplification without distortion.

7.3.2 Asynchronous Architecture Is Not Optional

Cloud orchestration tasks can take vastly different lengths of time to complete. For instance, creating a network, attaching a volume, waiting for an IP address, or dealing with provider metadata cannot be executed in the request-response loop without greatly compromising the user experience. Thus, the use of Celery and Redis was not merely an imperative from the performance perspective but a logical necessity.

7.3.3 Security Design Should Begin With Secret Boundaries

Credential handling rapidly becomes the most critical issue when building a multi-cloud controller plane. A universal dashboard has tremendous utility because it brings everything together under one roof, but doing so comes with its own risks. It became clear during the development process how important it is to have clear boundaries around the storage and usage of secrets, specifically: encrypt everything on ingress, store the encrypted values only, decrypt only in a minimal scope of execution, and structure the log messages such that no credential material can be recovered from them.

7.3.4 Observability Improves Design Quality

It should also be pointed out that instrumentation is not only beneficial after implementation; it actually helps optimize architecture. When task identifiers, log entries, state changes, and error types are exposed, architecture problems are more easily recognized. Retry storms, outdated inventories, slow provider invocations, and invalid payloads become evident, whereas before,

they may have been merely anecdotal concerns. Thus, observability serves as an engineering feedback loop in a similar fashion.

7.4 Limitations of the Present Work

It would be unrealistic to expect an academic endeavor to cover all aspects of cloud operations in its entirety, and this applies to the project at hand as well. There are several limitations inherent to the current state of the project which must be considered. First, the provider coverage is purposely limited. The system aims to address a subset of high-value infrastructure objects like compute, storage, and network primitives. Managed databases, identity federation, containerization systems, and event-driven systems receive only cursory treatment. Second, resource abstraction suffices for most cases but is provider-specific in others.

Third, while the AI copilot is helpful in diagnostics, it cannot provide absolute accuracy. It can only be as good as the log data fed to it, sanitization policies used, and clarity of prompts. In this regard, the final decision to make about remediation must still come from humans. Fourth, Terraform execution is done through isolated actions using local workspaces, which is appropriate for an academic prototype but may require better state management, policy adherence, and concurrent execution controls at the scale of a real-world deployment.

Lastly, while the evaluation done is more inclined towards checking architectural and process soundness, rather than assessing load at internet scale or enterprise certification, it does not mean that there is less value added by the Nebula prototype presented here. In actuality, Nebula is a practically oriented, soundly built multi-cloud infrastructure orchestration system prototype.

7.5 Academic and Industrial Relevance

This work is important to academia since it combines multiple concepts, which usually remain separate. Namely, this project considers distributed systems, Infrastructure as Code, secure software design, Human Computer Interaction, and applications of AI. Student projects usually only focus on some particular layer of the technology stack. Nebula differs because it looks at interactions between all these layers together. The performance of a control plane is not determined by the API routes alone but rather by the security characteristics of the credential flow, long-running task behavior, and user interaction feedback mechanisms.

Second, the project is also applicable in an industry setting since small and medium companies frequently use several different cloud providers without having a specialized platform team for each one. This means that the overhead of managing such a setup is no longer technological but rather cognitive. It takes engineers time to switch mental models and reconcile terminology in case of different providers. By adopting a workflow-first approach, Nebula

tackles this problem and saves time for onboarding, recovery, and maintenance.

7.6 Concluding Remarks

With Nebula, we have demonstrated that there is no reason why multi-cloud management has to stay fragmented, black-boxed, and overly complicated. Through the combination of deterministic infrastructure execution, asynchronous orchestration, and remediation guidance, the Nebula project serves as a compelling vision for what cloud management systems of tomorrow may look like, in terms of their architecture. The final takeaway is thus not just the implementation code snippets or the architectural diagrams provided above but, more importantly, the architectural argumentation behind them—namely, that a unified control plane can be made both secure, observable, extensible, and user-friendly with enough thought put into the design up front.

In summary, our conclusion could not be more straightforward. Multi-cloud management is more than a mere deployment model—it is a complex integration challenge. To overcome it, one needs a solid set of abstractions, proper management of secrets, robust background job execution, and clear operator guidance. Nebula offers a promising basis in all four aspects, while the upcoming chapters of this paper will explore further advances in each direction.

Chapter 8

ALGORITHM / MATHEMATICAL MODEL

For maximizing operational determinism and security, the Nebula platform utilizes specialized algorithms for some of its core components such as the AI Troubleshooting Copilot, the normalization layer of resources, the scheduling algorithm for retries, and the security vault for credentials. These models will be presented formally using pseudo-code, descriptions of states, and mathematical notation in this chapter. This chapter is not concerned with demonstrating theoretical novelty in all components; rather, it seeks to demonstrate that the control plane can be discussed from a theoretical perspective instead of only being described at a high-level.

8.1 The AI Copilot: Adaptive Log Remediation Algorithm

The AI Copilot was created specifically to address the semantic mismatch issue in multi-cloud error logs. In case of a Terraform task failure, the log becomes extremely long and provider-specific. ALR minimizes noise with deterministic preprocessing and adaptive inference techniques.

Assume the log from Terraform execution consists of token sequence

that could include provider-specific data, time stamps, stacks, resource IDs, and other confidential information. The objective of ALR is to transform this input into tuple

$$L = \{l_1, l_2, \ldots, l_n\}$$

where each token may include provider messages, timestamps, stack traces, resource identifiers, or sensitive values. The goal of the remediation pipeline is to produce a structured tuple

$$R = (c, s, a, \gamma)$$

where c is the failure category, s is a concise summary, a is a suggested action, and $\gamma \in [0, 1]$ is the confidence score.

The pipeline applies four stages:

1. **Sanitization**: remove tokens that may contain secrets, internal IPs, or organization-specific identifiers.
2. **Context enrichment**: attach resource type, provider, region, and task metadata.
3. **Classification**: identify likely failure families such as authentication, quota, syntax, dependency, or connectivity.
4. **Action mapping**: convert the detected family into a deterministic remediation recommendation where possible.

8.1.1 Pseudo-code for the ALR Algorithm

Listing 8.1: AI Copilot Remediation Logic

```
def execute_remediation_pipeline(task_id: int, raw_log: str) -> dict:
    # Stage 1: Scrubbing and anonymization
    scrubbed_log = scrub_secrets(raw_log)
    trimmed_log = prioritize_error_windows(scrubbed_log, max_lines=80)

    # Stage 2: Context injection
    ctx = get_task_context(task_id)
    prompt = build_prompt(
        provider=ctx.provider,
        resource_type=ctx.resource_type,
        operation=ctx.operation,
        region=ctx.region,
        log_excerpt=trimmed_log,
    )

    # Stage 3: Controlled inference
    llm_output = llm.query(prompt, temperature=0.1)

    # Stage 4: Deterministic post-processing
    category = classify_failure(llm_output, trimmed_log)
    remedy = map_category_to_remedy(category, ctx)
    confidence = score_confidence(category, llm_output, trimmed_log)

    return {
        "task_id": task_id,
        "category": category,
        "summary": llm_output.get("summary"),
```

```
        "remedy": remedy,
        "confidence": confidence,
        "auto_fix": category in {"quota", "missing_tag", "retryable_lock"
            },
    }
```

8.1.2 Confidence Scoring Model

The confidence measure must not be a fuzzy guideline. Nebula calculates the confidence measure using the following formula:

$$\gamma = \alpha p + \beta m + \delta h$$

where p refers to the probability provided by the classifier, m is the match weight of the log pattern with known error patterns, and h represents the historical success ratio of the recommended remediation action for similar faults. These weights adhere to the following constraint:

$$\alpha + \beta + \delta = 1$$

with $\alpha > \beta > \delta$ holds true under the default setting since present evidence is more important than past similarities.

8.2 Unified Resource Normalization Algorithm

A critical issue in the platform architecture lies in mapping the provider-specific meta-data into a normalized schema that both the frontend and the database can understand uniformly. Let the provider-specific resource response be denoted by Xp where X_p, where $p \in \{\text{AWS}, \text{Azure}, \text{GCP}\}$. The normalization function

$$f_p(X_p) = Y$$

maps this provider response into a canonical tuple

$$Y = (\text{id}, \text{name}, \text{provider}, \text{type}, \text{region}, \text{status}, \text{metadata})$$

used throughout the platform.

8.2.1 Pseudo-code for Resource Canonicalization

Listing 8.2: Provider Resource Normalization

```
def normalize_resource(provider: str, payload: dict) -> dict:
    if provider == "aws":
```

```
        return {
            "provider_id": payload.get("InstanceId") or payload.get("
                BucketName"),
            "name": extract_aws_name(payload),
            "provider": "AWS",
            "resource_type": infer_aws_type(payload),
            "region": payload.get("Region", "unknown"),
            "status": map_aws_state(payload),
            "metadata": payload,
        }
    if provider == "azure":
        return {
            "provider_id": payload.get("id"),
            "name": payload.get("name"),
            "provider": "Azure",
            "resource_type": infer_azure_type(payload),
            "region": payload.get("location", "unknown"),
            "status": map_azure_state(payload),
            "metadata": payload,
        }
    if provider == "gcp":
        return {
            "provider_id": payload.get("id") or payload.get("selfLink"),
            "name": payload.get("name"),
            "provider": "GCP",
            "resource_type": infer_gcp_type(payload),
            "region": extract_gcp_region(payload),
            "status": map_gcp_state(payload),
            "metadata": payload,
        }
    raise ValueError("Unsupported provider")
```

It is essential to introduce this normalization layer because it sets up the necessary boundary condition for the rest of the platform. In cases where canonicalization has already taken place, it does not matter anymore what kind of field was present in the cloud payload; it could have been something like `provisioningState`, `InstanceState`, or status.

8.3 Task Scheduling and Retry Model

The cloud environment, on its own, is not a reliable one in the short term, although the desired operation might be legitimate. The API can throttle, there will always be some kind of lock, capacity might be temporarily unavailable, and other services could still be initializing. This problem is solved using a bounded-retry approach within the Nebula framework.

Let a task T_j have an initial submission time t_0 and retry counter r_j. For retryable failures,

the next scheduled attempt is defined by exponential backoff:

$$t_{j,r+1} = t_{j,r} + b \cdot 2^r + \epsilon$$

where b is the base delay and ϵ is a jitter term used to avoid synchronized retries across many tasks.

8.3.1 Retry State Transition Table

This particular state model guarantees that the user interface is able to convey meaningful status. The resource is not just stuck; it might be queuing, in progress, in retry schedule, done, or failed. It is necessary in order to give the users reliable feedback while their task runs on the infrastructure that could take a few minutes to complete.

8.4 Mathematical Model for Credential Security

The security vault encrypts sensitive cloud credentials based on a symmetric approach. Here is how this could be represented in terms of cryptography notation.

8.4.1 Encryption Foundation

Let K be the unique, 128-bit master secret key stored as an environment variable. Let M be the plain-text cloud credential (e.g., Azure Client Secret). Let IV be a randomly generated Initial Vector (to prevent identical messages from having identical ciphertexts).

The encryption process E can be defined as:

$$C = E(K, M, IV)$$

where C is the output ciphertext. Our Nebula vault uses a symmetric construction wrapped around the Fernet interface. The main characteristic that our application requires from this system is storing only the ciphertext along with metadata in the database. For two identical credentials M1 = M2 being encrypted separately, randomized encryption guarantees

If two identical credentials $M_1 = M_2$ are encrypted independently, randomized encryption ensures

$$E(K, M_1, IV_1) \neq E(K, M_2, IV_2)$$

whenever $IV_1 \neq IV_2$. This prevents direct equality inference from stored ciphertext values.

[node distance=1.7cm, auto, scale=0.85, every node/.style=scale=0.85] block = [rectangle, draw, rounded corners, fill=blue!8, text width=7em, text centered, minimum height=3em] line = [draw, -latex’]
[block] (raw) Raw Terraform Log; [block, right=of raw] (scrub) Secret Scrubbing; [block, right=of scrub] (context) Context Injection; [block, right=of context] (llm) LLM Inference; [block, below=1.6cm of context] (classify) Category Mapping; [block, left=of classify] (confidence) Confidence Scoring; [block, right=of classify] (remedy) Structured Remedy;
[line] (raw) – (scrub); [line] (scrub) – (context); [line] (context) – (llm); [line] (llm) – (remedy); [line] (context) – (classify); [line] (llm) – (classify); [line] (classify) – (confidence); [line] (classify) – (remedy);

Figure 8.1: Adaptive Log Remediation Pipeline in the Nebula AI Copilot

Table 8.1: Task State Machine for the Nebula Worker

Current State	**Trigger**	**Next State**
queued	worker accepts task	running
running	terraform apply success	completed
running	retryable provider failure	retry_scheduled
running	non-retryable validation error	failed
retry_scheduled	backoff timer elapsed	running
failed	user submits corrected payload	queued

[node distance=3.5cm, auto, scale=2.9, every node/.style=scale=0.9] state = [rectangle, draw, rounded corners, fill=green!10, text centered, minimum width=7em, minimum height=2.6em] line = [draw, -latex’]
[state] (queued) Queued; [state, right=of queued] (running) Running; [state, right=of running] (completed) Completed; [state, below=of running] (retry) Retry Scheduled; [state, below=of completed] (failed) Failed;
[line] (queued) – (running); [line] (running) – (completed); [line] (running) – node[right] retryable error (retry); [line] (retry.south) -| node[below] backoff elapsed (running.south);
[line] (running) – node[right] hard failure (failed); [line] (failed.west) -| node[below] corrected request (queued.south);

Figure 8.2: Lifecycle of a Nebula Background Task

8.4.2 Decryption at Runtime (Worker Scope)

For Celery to be able to run the Terraform plan, it requires to unseal the credential M. The algorithm D looks as follows

$$M = D(K, C, IV)$$

This security property means that in the event of data theft by an attacker, the security of cloud accounts is much better protected since the key K will never be persisted in the database.

8.4.3 Operational Security Assumptions

To be more precise, compromising the storage system by itself does not mean compromising the secret itself since the attacker should also steal application-layer key material used to perform decryption.

1. the key K will never be hard-coded in the source code but delivered in a secure fashion via the runtime,
2. memory space used by a worker process can be considered trusted only within the scope of a particular job execution,
3. Decrypted secrets should never appear in execution logs,
4. Rotation of keys should invalidate previous encryptions.

These assumptions are what connect the notation to a secure implementation practice.

8.5 Complexity Analysis of Resource Syncing

Nebula needs to synchronize all users' resource inventories periodically in many clouds.

N is the number of cloud providers (AWS, Azure, and GCP), and Ri is the number of resources within the ith cloud provider.

T is the polling period.

The computation cost of a full synchronization cycle is O(ΣNi Ri). Because of the asynchronous task queue implementation, the delay from the user point of view is $O(1)$ compared to the synchronization process. In case the workload is distributed among W worker processes, the estimated synchronization time of a full cycle will be

The total computational complexity of a full sync cycle is $O(\sum_{i=1}^{N} R_i)$. Since we use an asynchronous task queue, the perceived latency for the user is $O(1)$ relative to the sync operation itself.

If the sync work is parallelized across W worker slots, the idealized processing time for a single cycle becomes

$$\Theta\left(\frac{\sum_{i=1}^{N} R_i}{W}\right)$$

subject to provider API rate limits and serialization points such as database writes. In practice, the observed time is bounded below by the slowest provider batch and bounded above by the total sequential scan.

It depends on the API rate limiting of providers and serialization points like writing to databases. Practically speaking, the time taken will be bound from below by the longest provider batch and from above by the total sequential scan.

8.6 Decision Function for Cloud Selection

While the existing system lets the user manually pick the desired provider, there is also support for an advisory-based selection process in which the system computes a score for a potential provider p

$$S(p) = w_c C(p) + w_l L(p) + w_r R(p) + w_s Sec(p) + w_a A(p)$$

where:

- $C(p)$ is the normalized cost score,
- $L(p)$ is the latency or proximity score,
- $R(p)$ is the regulatory fit score,
- $Sec(p)$ is the security-policy compatibility score,
- $A(p)$ is the availability and capacity score.

The weights used for this expression can depend on the type of workload. For a regulated workload, the w_r and w_s might dominate while, for experimentation, w_c and $L(p)$ might hold more sway. It is useful in the sense that all the future automation capabilities translate to an optimization problem.

8.7 Discussion

The models presented in this chapter make the workings of the Nebula core engine clearer. The AI copilot is cast as a bounded classification and mapping task, the resource inventory layer is formulated as a canonicalization function, retrying is modeled as a stateful backoff schedule, and credential management is depicted as controlled encryption and decryption across a secret

key boundary. Formal models like these improve the testability, analysis, and extensibility of the system, while at the same time making a clear connection between the objectives of the blackbook and the implementation chapters.

Chapter 9

FUTURE SCOPE AND ETHICAL CONSIDERATIONS

The evolution of the cloud computing field means that the ”Unified Command Center” paradigm defined by Nebula has laid out an initial groundwork for better orchestration techniques. The present setup has demonstrated that a reliable, asynchronous, and multicloud based control plane is possible. Now the key issue becomes what further developments could realistically build on the initial groundwork and what ethical considerations will need to be taken into account when defining such development paths. The future is not about just developing new functions, but also about how to behave when performing these functions.

9.1 Future Research and Enhancements

Though Nebula already provides management services for basic cloud infrastructure components such as virtual machines, storage, and networking primitives, its next significant step might be unification of the management of serverless computing functions. One ”Nebula Function” should be able to provide all necessary resources needed for its deployment on platforms like AWS Lambda, Azure Functions, and Google Cloud Functions. The benefit from it will be especially high for those organizations that need regional redundancy without having separate deployment pipelines for each one.

9.1.1 Serverless Cross-Cloud Orchestration (SaaS-less)

While Nebula currently manages core infrastructure such as virtual machines, storage, and networking primitives, a major future expansion is unified management of serverless functions. A single ”Nebula Function” could be expressed once and then translated into deployment artifacts for AWS Lambda, Azure Functions, and Google Cloud Functions. This capability would be especially valuable for teams that want regional resilience without maintaining multiple

provider-specific deployment pipelines. A follow-up research problem here is how to normalize differences in trigger models, packaging constraints, timeout policies, and cold-start behavior across clouds without creating a misleading abstraction.

9.1.2 Auto-Failover and Disaster Recovery (DR)

The next more sophisticated one would be "Zero-Touch Failover." In case of degradation of a whole region or even of an entire family of services, Nebula can launch Terraform plan that will reconstruct necessary infrastructure in an alternative provider/region altogether. This should be taken not as a guarantee of 100% uptime. Disaster Recovery entails tradeoffs when it comes to data freshness, DNS propaga- tion time, configuration divergence, and other legal issues. Thus, a serious Disaster Recovery system would include several levels of failover including cold site, warm site, and active-active site, among others. And Nebula, once again, would fit in perfectly since it already possesses the pro- visioning cycle process.

9.1.3 AI-Powered Cost Optimization

In terms of the current AI Copilot, its focus is failure analysis. In future iterations, it can extend to economic analysis through idle resource detection, burst profiling, storage capacity expansion, and compute underutilization detection. Instead of serving as an all-purpose chatbot, it can serve as a FinOps copilot that will explain why a workload is costly and how a different approach to deployment will lower costs. Examples of such advice would be to use spot-optimized regions for background tasks, replace overprovisioned virtual machine instances with optimal ones, and implement lifecycle management of rarely accessed objects.

9.1.4 Kubernetes and Managed Container Platforms

A related direction would involve having policies enforced prior to infrastruc- ture modifications. A policy engine would enforce region-based restrictions, mandatory tags, encryption requirements, naming standards, and network exposure policies. The end result would make Nebula more than an execution plane; it would be a control plane aware of governance concerns. From an academic perspective, policy enforcement actions would include logging along with reasons for enforcement, ensuring that when a user's deployment fails, they know not only that it failed, but why.

9.1.5 Policy-as-Code and Compliance Guardrails

First, the capability provided by such an orchestration layer implies certain ethical obligations, which follow from the nature of the capabilities offered. Specifically, the orchestration layer enables one to provision, manipulate and destroy cloud resources on several clouds. In other

words, the orchestration layer is anything but neutral in terms of ethical implications, and hence the degree of protection it offers should be considered an integral part of the layer's ethics.

9.1.6 Edge and Hybrid Resource Awareness

Organizations do not exist solely within the confines of the public cloud. Branch offices, factory machinery, research environments, and private virtualization systems all play their roles. A mature Nebula implementation would eventually move into hy- brid and edge environments through integration with agents or connectors reporting on local infrastructure to fit into the same canonical inventory framework. While difficult, this approach is essential because it will test the control-plane abstrac- tions created in this project beyond the public cloud environment.

9.2 Ethical and Security Considerations

First, the capability provided by such an orchestration layer implies certain ethical obligations, which follow from the nature of the capabilities offered. Specifically, the orchestration layer enables one to provision, manipulate and destroy cloud resources on several clouds. In other words, the orchestration layer is anything but neutral in terms of ethical implications, and hence the degree of protection it offers should be considered an integral part of the layer's ethics.

9.2.1 Data Sovereignty and GDPR

One of the most important ethical issues associated with using Nebula in the multi-cloud context relates to data residency. The application has to ensure that any sensitive information remains only in those geographic locations that are authorized. Simply putting the region option in the form of a dropdown menu does not help here since users may choose the wrong value. Instead, some geographic limitations should be embedded at the orchestration layer level, preventing the deployment of unauthorized plans.

9.2.2 The Power of a Unified API

Concentration of cloud access keys to one platform presents a high-value target for the attacker. Hence, the role of the developer ethically is to mitigate this risk of concentration. Runtime isolation, key rotation, use of credentials based on the principle of least privilege, log scrubbing, and audit trail are the ways of accomplishing it. It is imperative that the user knows which request action he made, what provider got affected, which credential and roles got impacted, etc.

9.2.3 AI Explainability and Human Oversight

In case the platform is equipped with artificial intelligence-driven recommendations, these recommendations must be explainable to the operator to make sure he has enough visibility to justify the applicability of such a recommendation. Any recommendation that cannot be justified from the log stream or by referring to a set of policies cannot be acted upon blindly by the operator. Failure scenarios are prone to this risk even more.

9.2.4 Cost Fairness and Organizational Transparency

The choice of cloud service providers and locations is not without organizational repercussions. An orchestration platform that prioritizes one vendor or location over another must disclose its decision-making process, which may include performance requirements, compliance issues, and cost considerations. In this context, ethical design implies transparency regarding the factors considered when making the decision.

9.2.5 Environmental Sustainability

Abstraction through cloud computing could unintentionally alienate cloud users from their responsibility to the environment. One possible improvement for the future would be a sustainability metric that can quantify the cost associated with over-provisioned, unused, or redundant infrastructure. Even though it will remain an estimate, it will bring users' attention to the issue of energy consumption by the cloud infrastructure.

9.3 Governance Guidelines for Responsible Operation

In order to responsibly implement a platform such as Nebula, it is necessary to adhere to the following governance principles:

1. **Approval boundaries**: destructive operations should require explicit confirmation or policy approval.
2. **Least privilege**: credentials for providers should be limited only to the resources and activities that are necessary.
3. **Auditable actions**: every provisioning, deletion, and remediation event should be attributable to a user or automation identity.
4. **Policy transparency**: enforcement rules should be documented and visible to operators.
5. **Human-in-the-loop AI**: automated suggestions should remain reviewable before irreversible changes occur.

[node distance=2.0cm, auto, scale=0.82, every node/.style=scale=0.82] core = [rectangle, draw, rounded corners, fill=blue!10, text centered, minimum width=10em, minimum height=3em] future = [rectangle, draw, rounded corners, fill=orange!10, text centered, minimum width=9em, minimum height=2.8em] line = [draw, -latex’]
[core] (nebula) Current Nebula Core; [future, above left=of nebula] (serverless) Serverless Control; [future, above=of nebula] (dr) Disaster Recovery; [future, above right=of nebula] (finops) AI Cost Optimization; [future, below left=of nebula] (k8s) Kubernetes Support; [future, below=of nebula] (policy) Policy as Code; [future, below right=of nebula] (hybrid) Hybrid / Edge Ops;
[line] (nebula) – (serverless); [line] (nebula) – (dr); [line] (nebula) – (finops); [line] (nebula) – (k8s); [line] (nebula) – (policy); [line] (nebula) – (hybrid);

Figure 9.1: Future Roadmap Dimensions for the Nebula Control Plane

Chapter 10

DEPLOYMENT AND OPERATIONS PLAYBOOK

The present chapter converts the system design into actionable steps. While the appendix user manual addresses itself to local configurations and usage, the present chapter deals with the environment architecture, release management, operational best practices, and support processes. An important practical question answered by the present chapter is how to maintain a reliable Nebula once it exists.

10.1 Environment Topology

The best way to operate `Nebula` is to do so in a multi-environment setup that has controlled environment promotions. The recommended base configuration includes:

- **Local development**: individual developer machines using mocked/sandboxed credentials.
- **Shared integration**: an environment to test APIs/UI/worker compatibilities.
- **Staging**: a near-production configuration with realistic secrets management and sandbox clouds.
- **Production**: actual live control plane using authenticated operators.

Different database, broker, and encryption key usage should be maintained per environment.

10.2 Release Pipeline

The pipeline for code releases needs to have the following stages:

1. static code validation of backend, frontend and Terraform files,
2. unit testing of the validation code, API operations and utility functions,
3. integration testing of queue workflow, data persistence to database,
4. sandbox deployment using non-production cloud credentials,
5. sandboxing with non-production credentials in cloud services provider environment,
6. security review of release notes for any impact on security,
7. gradual deployment to staging and production environments.

10.2.1 Illustrative CI/CD Flow

Listing 10.1: Release Pipeline Stages in Pseudo Workflow Form

```
name: nebula-release
on:
  push:
    branches: [main]

jobs:
  validate:
    steps:
      - checkout
      - run backend lint
      - run frontend lint
      - run terraform validate

  test:
    steps:
      - run unit tests
      - run API contract tests
      - run queue integration tests

  package:
    steps:
      - build api image
      - build worker image
      - publish versioned artifacts

  deploy-staging:
    needs: [validate, test, package]
    steps:
      - apply staging configuration
```

```
      - run smoke tests

  approve-production:
    needs: [deploy-staging]
    steps:
      - manual approval

  deploy-production:
    needs: [approve-production]
    steps:
      - roll out api
      - roll out workers
      - verify health endpoints
```

10.3 Operational Readiness Checklist

Before enabling the platform for active user traffic, the following readiness checks should be completed:

10.4 Observability and Service Levels

The operations team would require more information than logs from containers. In a good deployment, the system would output:

- latency stats for APIs,
- throughput of workers and retries,
- failure reasons grouped by provider,
- time to reconciliation,
- trends in authentication failures,
- saturation of database connections,
- length of queue over time.

Based on all this data, we can construct some basic SLO framework. For instance, interactive API calls need to have low latency, queue tasks need to be executed with certain guarantees, and failing tasks must provide an explainable reason. All this is achievable and relevant for SLOs in control plane applications.

[node distance=2.0cm, auto, scale=0.78, every node/.style=scale=0.78] env = [rectangle, draw, rounded corners, fill=green!10, text centered, minimum width=8em, minimum height=3em] line = [draw, -latex’]
[env] (local) Local Dev; [env, right=of local] (integration) Shared Integration; [env, right=of integration] (staging) Staging; [env, right=of staging] (production) Production;
[line] (local) – node[above] merge / validate (integration); [line] (integration) – node[above] smoke tests (staging); [line] (staging) – node[above] manual approval (production);

Figure 10.1: Recommended Environment Promotion Flow for Nebula Deployments

Table 10.1: Operational Readiness Checklist

Control Item	**Status Type**	**Expected Evidence**
API health endpoint	mandatory	returns HTTP 200 with version metadata
Worker queue connectivity	mandatory	successful enqueue and dequeue test
Database migrations	mandatory	schema version matches release tag
Secret injection	mandatory	runtime variables resolved without fallback defaults
Cloud sandbox credential test	recommended	read-only identity check passes for each provider
Audit logging	mandatory	create, delete, and login events persisted
Backup job verification	recommended	last successful snapshot within policy window

10.5 Backup, Recovery, and Rollback

Furthermore, production operations require recovery plans. The following items would need protection:

1. the PostgreSQL database state - users, roles, resources and task info,
2. configuration files and templates for Terraform modules,
3. images and releases,
4. logs and other operational data used in debugging.

Rollbacks need special handling at each layer. Rollback in applications could entail rolling back the API and workers back to an earlier image. Rollback in databases can be tricky since it involves making forward changes instead of backwards ones.

In addition, Terraform rollbacks can be difficult since the infrastructure state has possibly been modified. Therefore, the best way to do a deployment is through progressive releases accompanied by smoke tests rather than large production releases.

10.6 Incident Handling Model

In the case of a user impact incident, the following needs to happen:

document remediation steps and post-incident actions.

1. Classify the incident as either authentication problem,
2. provider outage, queues, database issues, or application regression.
3. Determine whether the blast radius involves single users, providers, or the entire system.
4. Collect diagnostic information like task logs and recent configuration changes.
5. Stabilize the system through halting any dangerous automations.
6. Communicate with users on the situation.
7. Document remediation steps and post-incident actions.

10.6.1 Example Runbook Entry

10.7 Change Management and Team Workflow

Running a multi-cloud control plane requires diligent change management because any bug could impact more than one cloud provider simultaneously. Therefore, the following distinctions must be made when managing changes in software:

- **feature changes**, that introduce new capabilities, like new resource types,
- **operation changes**, that include changes to logging, retry,
- **security releases**, concurrency, and observability,
- **security changes**, including anything involving authentication, credentials, and provider permissions.

The differences are important for planning how much review and testing is needed. For instance, changes to the retry scheduler should lead to stress testing, while changes to credentials should involve thorough security reviews.

10.8 Why This Playbook Matters

A lot of student projects usually end up with architectural diagram analysis and source code description. In practice, however, operational discipline will decide on survival of architecture when faced with real users. This chapter goes further than that contribution of the blackbook as it demonstrates that the platform was evaluated not only as an implementation task, but also as a service to be operated and improved upon.

Table 10.2: Sample Incident Runbook

Incident Type	**Immediate Action**	**Follow-up Action**
Queue backlog	scale workers, inspect stuck tasks	tune retry policy and concurrency
Credential decrypt failure	stop execution for affected tenant	rotate key material and re-encrypt secrets
Provider API throttling	enable backoff and reduce fan-out	review sync scheduling policy
Database saturation	shed non-critical jobs	optimize queries and add pooling controls

Chapter 11

IN-DEPTH CLOUD PROVIDER COMPARISON

To make a useful multi-cloud platform one needs to know about technological details and mappings for each of three large cloud providers, including Amazon Web Services, Microsoft Azure, and Google Cloud Platform (GCP). The last point is critical since the abstraction layer of Nebula should be useful enough and cannot assume that cloud providers do not have any specific features that would allow them to be distinguished from each other. This chapter compares cloud providers based on seven different criteria: compute, storage, networking, identity, platform services, operations, and decision support.

11.1 Compute Resource Mapping (Virtual Machines)

Cloud compute services are grouped based on the type of their instances, such as general-purpose, compute-optimized, memory-optimized, and burstable. Similarly, nebula categorizes the instances in a single size model to enable capturing intention at a broad level, along with specifics from a particular provider.

11.1.1 Standard Instance Type Comparison

11.1.2 Hypervisor and Architecture Differences

- **AWS Nitro System**: AWS uses a specially designed Nitro hypervisor that moves network and storage traffic to special hardware, thus improving performance consistency.
- **Azure Hyper-V**: Microsoft makes use of a proven Hyper-V stack that is closely integrated with the Windows OS.
- **GCP KVM**: GCP uses a specially optimized version of the KVM hypervisor.

11.1.3 Burst Behavior and Instance Positioning

Though instances have the same sizes, there could be differences between them when it comes to bursting capabilities and default operations. The T-series instances in AWS and B-series instances in Azure use a credit-based approach that works well for applications that require bursting. E2 shared-core instances in GCP work best for applications that have a constrained budget, but they could behave differently under continuous pressure. What concerns Nebula, even though the creation of a common micro class would make it easier to unify the API, it is still important to inform users if an application has outgrown burstable instances.

11.2 Object Storage Performance and SLAs

Cloud vendors offer object storage with the same amount of durability, but the differences come from names, redundancy and lifecycle options. When it comes to orchestration, object storage is one of the easiest services to standardize due to the simplicity of underlying concepts – creation, listing, uploading and lifecycle.

11.2.1 Object Storage Durability Table

11.2.2 Storage Class Translation

Tiering is a good tool to estimate recommendations and lifecycles, but pricing, access costs, latency and redundancy are quite variable. They should be presented as separate views.

11.3 Network Topology and Addressing

Although the vocabulary may be different, the concepts that govern both cloud companies are largely similar. In the example of Nebula, the following is the definition for the concept known as ”network boundary”:

11.3.1 Terminology Translation Matrix

- **VPC (AWS)**: Virtual Private Cloud.
- **VNet (Azure)**: Virtual Network.
- **VPC Network (GCP)**: A global resource in Google Cloud.

11.3.2 Routing and Peering Mechanisms

1. **AWS VPC Peering**: A way of connecting two VPCs through their private IP addresses. Only region-specific, unless there is inter-region connection enabled.

2. **Azure VNet Peering**: A process of connecting two virtual networks through the high-speed and low latency backbone network of Azure.

3. **GCP VPC Peering**: GCP cloud network is a global network infrastructure, which gives it an edge over AWS and Azure.

11.3.3 Implication for Orchestration

Networking is one of the best examples of why multi-cloud abstraction should be approached with caution. While AWS has its VPC, Azure has VNet, and Google Cloud has its VPC network, "network" is simply an implementation of the idea of the logical network boundary. However, their default settings, scope, and other services can be quite different from each other. Therefore, there should be a standard operator's view of the cloud network in Nebula.

11.4 Identity and Access Management

Identity is typically considered the hardest domain to unify because of different implementations.

In the case of Nebula, this aspect becomes relevant since credential management and least privilege must differ depending on the chosen provider. Having a generic credential wizard will probably not be helpful. A more practical approach might involve providing a set of recommendations specific to the selected provider for managing credentials and saving authorization information via the common control plane API schema.

11.5 Managed Data and State Services

While the initial example uses lower-level primitive concepts for the infrastructure, practical use involves many managed forms of data and services. Among them there are some such as:

- **Relational databases**: Amazon RDS, Azure SQL/Azure Database service, Cloud SQL.
- **Managed caches**: Amazon ElastiCache, Azure Cache for Redis, Memorestor.
- **Message brokering**: Amazon SQS/SNS, Azure Service Bus, Google Pub/Sub.

Table 11.1: Cross-Cloud Compute Resource Mapping

Nebula Tier	vCPU/RAM	AWS Instance	Azure VM	GCP Machine
Micro	1 vCPU, 1GB	t3.micro	Standard_B1s	e2-micro
Small	2 vCPU, 2GB	t3.small	Standard_B2s	e2-small
Medium	2 vCPU, 4GB	t3.medium	Standard_B2ms	e2-medium
Large	4 vCPU, 8GB	t3.large	Standard_B4ms	e2-standard-4

Table 11.2: Cross-Cloud Object Storage Durability and Availability

Feature	AWS S3	Azure Blob	Google GCS
Durability	99.999999999%	99.999999999%	99.999999999%
SLA (Availability)	99.9%	99.99%	99.95%
Versioning Support	Yes	Yes	Yes
Lifecycle Policies	Yes	Yes	Yes

Table 11.3: Approximate Mapping of Storage Access Tiers

AWS S3	Azure Blob	Google Cloud Storage
Standard	Hot	Standard
Standard-IA	Cool	Nearline
One Zone-IA	Cool with local constraints	Regional reduced-access pattern
Glacier / Deep Archive	Archive	Coldline / Archive

Table 11.4: Identity Model Comparison

Aspect	AWS	Azure	GCP
Primary model	IAM users, roles, policies	Entra ID identities with RBAC	IAM principals and role bindings
Service identity style	IAM role / access key	Service principal / managed identity	Service account
Policy granularity	JSON policy documents	role assignments on scopes	resource hierarchy bindings
Common operator challenge	policy verbosity	tenant and subscription separation	project and folder hierarchy complexity

Importantly, the relevance of these cloud services lies in the obvious recognition that the more abstract the service becomes, the more difficult becomes the enforcement of the mapping among multiple clouds. As such, Nebula must treat these as capability families, not equate them across multiple clouds.

11.6 Container and Kubernetes Ecosystem

The next-generation multicloud architecture revolves around containers and managed Kubernetes clusters. For AWS, it is EKS; for Azure AKS; for GCP, it is GKE. They all share the same capabilities such as managed control plane, node pool, ingress mechanisms, and scaling, yet their operational ergonomics differ.

GKE enjoys the best Kubernetes experience since Google has extensive experience working both with containers and Kubernetes. AKS will be preferred in the Microsoft environment due to compatibility with Azure Identity Services and Network Management Services. EKS is considered valuable for its integration into other AWS services as well as its wide production adoption. Therefore, in case of Nebula's provision of the container orchestrator support, general lifecycle tasks would matter more than any similarities: cluster enumeration, node scaling, workload rollout status, and policy visualization.

11.7 Operations, Monitoring, and Support Tooling

Each of the platforms also highlights its mature, albeit unique, observability stack:

- **AWS**: CloudWatch, CloudTrail, Config, and X-Ray.
- **Azure**: Azure Monitor, Log Analytics, Activity Log, and Application Insights.
- **GCP**: Cloud Monitoring, Cloud Logging, and Cloud Audit Logs within Google Cloud Operations.

While these native services are valuable, at the same time, they are part of the fragmentation problem that Nebula tries to address. There is no need for a control plane to fully replace these services. However, what it can do is gather decision-relevant signals from those services, such as resource health, deployment state, classification of failures, and compliance drifts.

11.8 Pricing and Billing Behavior

Pricing comparisons based purely on list prices are often misleading because pricing depends on regions, reservations, sustained use, egress, storage retrieval, and support. In terms of orchestration needs, the following comparisons can be made in terms of:

1. burstable development environments,
2. archival workloads heavy on storage,
3. analytics or machine-learning bursts,
4. disaster recovery standby environment across multiple regions,
5. steady-state web services.

Nebula can add value here by expressing cost guidance in the context of these workload shapes rather than exposing raw price sheets with little interpretation.

11.9 Provider Selection Matrix for Nebula

11.10 Multi-Cloud Strategic Vendor Benchmarking

Nebula provides users with a ”Cloud Scorecard” based on the following criteria:

- **Service Breadth**: AWS has the largest range of services - 200+ native services.
- **Enterprise Integration**: Azure is used primarily by those who use Active Directory or MS Office 365 solutions.
- **Big Data/AI Performance**: GCP is considered a leader in the field of artificial intelligence and big data.

It would be incorrect to assume that the best choice lies with one particular cloud. Each of them is optimized to work around a specific center of gravity. AWS focuses on breadth and service maturity, Azure focuses on integration and organization, and GCP focuses on analytical capabilities and ease of working with new technologies. The importance of Nebula is that it understands that many companies require more than just one of the advantages mentioned above.

Table 11.5: Illustrative Provider Suitability Matrix

Workload Type	**AWS**	**Azure**	**GCP**
General enterprise application	High	High	Medium
Microsoft-centered enterprise stack	Medium	Very High	Medium
Analytics-heavy or data science experimentation	High	Medium	Very High
Broad service portfolio requirement	Very High	High	Medium
Kubernetes-first platform team	High	High	Very High

Chapter 12

DEVOPS PIPELINE AND RELEASE ENGINEERING

In addition to being a piece of software, Nebula also constitutes a way of deploying the software. As such, the surrounding DevOps pipeline falls within the technical scope of the project. The infrastructure manager that governs multiple clouds also needs to be deployed through a reliable, reproducible, and low-risk release cycle. This chapter discusses the release engineering principles behind that endeavor.

12.1 Source Control and Branching Strategy

Code management benefits greatly from a branching policy that distinguishes development and release activities. An efficient structure consists of:

- **main**: branch for production releases with merge protection,
- **develop**: branch for integrating finished features,
- **feature/***: task-specific branches,
- **hotfix/***: emergency fixes for critical production problems.

The strategy minimizes risk during the release cycle by making it possible to review infrastructure changes, frontend changes, and backend changes simultaneously prior to deploying into a shared environment. Traceability is ensured since each release can be traced to its source code, pull requests, and approvals.

12.2 Build Artifacts and Versioning

Nebula creates several artifacts ready for deployment:

1. frontend static resources,
2. backend API service image,
3. worker container image,
4. Terraform module template and shared configuration bundle, database migrations scripts.
5. database migration scripts.

All these elements need to be released under consistent versioning using a specific tag, such as `v1.4.0`. The important point here is to align the versions of the API service image, worker service image, and the database migration status against the same release. Otherwise, the debugging process will become unnecessarily complex because various elements may be based on conflicting assumptions.

12.3 Continuous Integration Stages

The CI process serves as the first filter for ensuring software quality. In its minimal implementation, the continuous integration tool should run:

- backend static analysis and unit testing,
- frontend linting and build verification,
- syntax validation of Terraform templates,
- migration schema validation,
- API contract or smoke test.

The interesting aspect of this approach is that Terraform templates are considered as code artifacts. Although both the backend services and front end applications have passed all necessary tests, incorrectly structured Terraform template can make the entire platform unusable. Thus, the IaC layer must be tested along with the application layer.

12.3.1 Illustrative Pipeline Definition

Listing 12.1: Representative CI Pipeline Skeleton

```
stages:
  - validate
  - test
  - package
  - deploy
```

```

validate_backend:
  stage: validate
  script:
    - python -m pytest tests/unit

validate_frontend:
  stage: validate
  script:
    - npm ci
    - npm run build

validate_iac:
  stage: validate
  script:
    - terraform fmt -check
    - terraform validate

package_api:
  stage: package
  script:
    - docker build -t nebula-api:$CI_COMMIT_TAG backend

package_worker:
  stage: package
  script:
    - docker build -t nebula-worker:$CI_COMMIT_TAG backend
```

12.4 Environment Promotion Workflow

Releases must advance from environments that are less risky to more risky. The typical approach is to:

It is critical that releases advance like this because it is possible for multi-cloud control plane releases to have issues that regular unit tests will not discover. This is because when deploying, problems may arise during the following scenarios:

1. merge reviewed code into the integration branch,
2. automatically deploy to a shared test environment,
3. run smoke tests on login, queuing, and provider connection,
4. deploy the same artifact to the staging environment,
5. approving deployment to production,

6. closely watch early metrics after deploying to production.

This progression matters because a multi-cloud control plane can fail in subtle ways that ordinary unit tests do not catch. A release may look healthy until it attempts provider authentication, state reconciliation, or asynchronous task execution with live infrastructure APIs. Promotion through increasingly realistic environments is therefore a core risk-reduction strategy.

12.5 Secrets and Configuration Management

The release pipeline should know how to separate code from environment-dependent sensitive data. The following list contains items that should never be kept in source control:

The release pipeline must distinguish between code and environment-specific secret material. The following items should never be embedded in source control:

- master encryption keys,
- production database credentials,
- service principal password,
- third-party API tokens.

These configuration parameters should be injected at runtime via environmen- tal variables or secret management systems. This will help with secure rotation, minimize risk of leakage and maintain portability of the codebase across environments.

12.6 Containerization Strategy

Containerization will bring value to the Nebula application since it will guaran- tee consistent execution environment. API, workers, databases, and brokers are composable components that can be reproduced by developers on their local environments. Benefits include:

- consistent runtime dependencies,
- easy onboarding process,
- versioned artifact deployments,
- defined boundaries of services,
- rollback mechanism to known good versions.

Containerization is not enough to ensure operational reliability. Container images require additional review of vulnerabilities, health checks, startup probes, and proper logging. It is particularly critical for the worker container which uses secrets and manages the execution of Terraform.

12.7 Database Migration Discipline

Migration of schemas is another frequent cause of fragile releases. Nebula holds information about users, credentials, tasks, and resource states; an irresponsible migration could impact security and availability. In such a case, a well-thought-out migration strategy should consider:

1. migrating forward only when possible,
2. reviewing nullable-to-non-nullable changes explicitly,
3. performing data backfills as release steps,
4. backing up data before any destructive migration steps,
5. conducting smoke tests on the migrated schema.

12.8 Rollback Strategy

Each part of a release has its rollback strategies:

- **API code rollback**: rdeploying previous versions of API and worker images.
- **Configuration rollback**: reverting to proven configurations and feature flags.
- **Infrastructure rollback**: mplementing reverse changes from a thorough review, not assuming immediate cloud infrastructure rollback.

The difference between the three kinds of rollback is important since it might be that the release failed in one domain but succeeded in others. For instance, the API code is stable enough to be rolled back, whereas the database schema still supports backward compatibility.

12.9 Release Readiness Checklist

12.10 Discussion

DevOps pipeline is no different from any other element that comprises the product itself; this element constitutes an integral part of the reliability envelope of the product. The idea behind

Nebula is to decrease the complexity associated with operations for the sake of its users. For the same reason, the development team needs to practice discipline while building the very platform itself by applying such measures as versioning of artifacts, validation of infrastructure templates, use of secrets management outside the source code, and release promotion across multiple stages.

Table 12.1: Release Gate Criteria

Check	**Required**	**Rationale**
Unit and integration tests pass	Yes	baseline quality signal
Terraform templates validate	Yes	prevents broken provisioning flows
Migration reviewed and backed up	Yes	protects persistent state
Secrets resolved in target environment	Yes	avoids startup and runtime failures
Staging smoke test completed	Yes	confirms end-to-end functionality
Operational owner assigned	Yes	ensures release accountability
Rollback note prepared	Recommended	accelerates incident response

Chapter 13

USE CASES, CASE STUDIES, AND COST SCENARIOS

Multi-cloud orchestration becomes more valuable when assessed in terms of practical use case applications. This chapter will present several representative examples to highlight how Nebula can be used for various needs. There are no claims about universality and perfection made, instead, it aims to illustrate the power of a unified control plane as an abstraction for decision-making, deployment choices, and financial considerations.

13.1 Use Case 1: Startup SaaS Platform

Assume a startup working on customer-facing SaaS product and having a small engineer- ing group. The goal is to develop the solution fast and clearly, minimize upfront costs, and refrain from getting hooked up with one specific cloud provider for now.

In such a situation, Nebula can assist with experimentation by offering a standardized interface for developing VMs, object stores, regional instances, etc. Instead of having to get used to all three portals immediately, the company will be able to make standardized requests and compare results side-by-side.

The key value here is cognitive efficiency. The engineering team is typically small and consists of people who both implement the solution and provide the corresponding infrastructure. Nebula will help reduce cognitive load, facilitate onboarding, and retain knowledge of what was deployed where and why.

For this type of organization, the main benefit is cognitive efficiency. The team is usually small, and the same engineers often handle both product features and infrastructure work. Nebula reduces context switching, shortens onboarding time for new team members, and preserves a record of what was provisioned and why.

13.2 Use Case 2: Enterprise Department With Microsoft Alignment

Assume an enterprise department that heavily utilizes Microsoft-related services for their identity and collaboration needs but wants certain analytics or external-facing solutions to be deployed in other clouds. For example, there might be an interest in leveraging AWS/GCP as opposed to Azure because the workload is better suited to particular providers. In such a scenario, Nebula can serve as a tool to keep Azure-based policy framework while still being able to manage other services in controlled manner.

In such an environment, multi-cloud orchestration is mostly about the policy-boundary nature as opposed to merely being convenient. The importance of onboarding, tag enforcing, restriction to regions, and auditability cannot be underestimated. However, having the same dashboard remains important because the department should know where resources are hosted regardless of the need for everyone to be proficient in each provider's console. e.

13.3 Use Case 3: Research Laboratory or Academic Institution

Another scenario in which Nebula will work efficiently is when there are periodic demands for a computing cluster, storage facility, and reproducibility of the infrastructures for academic assignments or experiments. In such cases, budget visibility and repeatability will help since new people join the group and leave after some time, thereby making infrastructure information fragment among different groups of students.

The ability of Nebula to document, observe, and reuse workflows suits such scenarios well. This is because the control plane allows a technical lead to define a series of standardized procedures for setup and provide infrastructure templates for them to follow. Moreover, the technical leads will be able to see all the components and what has been set up using various providers.

13.4 Use Case 4: Seasonal E-Commerce Expansion

In the case where e-commerce sites experience seasonal demands, it might be desirable to maintain one cloud provider while being ready to use a second one in case the first one fails. In most times, not all resources will be used at any time depending on the level of activity. However, during high activity levels, it might be desirable to use the second cloud as well.

In such scenarios, the team must know how to distinguish between primary and backup resources, how resource inventory can be kept synchronized, and how faults can be analyzed

quickly. This is one of the challenges the control plane can address by allowing task tracking and centralized inventory viewing.

13.5 Illustrative Cost Scenario Comparison

The following table provides an example of the kind of workload-based decision-making the control plane will allow. The numbers used in the table below are illustrative and not vendor quotes.

13.6 Case Study: Failed Provisioning With Guided Recovery

One of the most common examples of how useful Nebula is in practical terms would be an error at the moment of a time-bound release, such as a deployment failing due to a permissions error in AWS, which is detected via Terraform. In a traditional case, the operator has to decipher the log output of the provider, determine what permissions are missing, access the IAM console separately and find out if the issue is related to credentials, scope, or regions.

In the case of Nebula, on the contrary, the operator would be presented with the record of the task completed with a status indicating a failure, the log output would be saved in one place, while the AI copilot will analyze the log and make suggestions as to why the deployment failed. Regardless of the accuracy of these recommendations, in most cases, narrowing down the search space will save additional time for the operator, which is crucial in stressful situations.

13.7 Case Study: Inventory Drift Detection

The other issue, which is common in any cloud provider's environment, is called inventory drift. It usually happens when resources are manually deleted via a provider portal or modified without following the automated deployment path. That leads to a situation when local inventory and real resources in the cloud have different state. With Nebula's periodic synchronization procedure, it is easier to bring the two back into agreement by reconciling provider state with the database.

In case a company works with several clouds simultaneously, such functionality will pay off many times over by providing accurate information about cloud estate in one central place.

13.8 Organizational Benefits Beyond Provisioning

The long-term value of a system like Nebula is not limited to provisioning speed. It also creates benefits in:

- **training**: new engineers learn one operational workflow first,
- **documentation**: a single report or manual can describe cross-cloud actions consistently,
- **governance**: audit trails and approval patterns become easier to define,
- **support**: failures are easier to triage through centralized task records,
- **planning**: teams can compare provider choices from a shared interface.

13.9 Limitations of Scenario-Based Evaluation

The above examples are meant to be representative rather than exhaustive. Actual organizations might face other complications, such as different security boundaries, network topology issues, data privacy concerns, or procurement regulations that our academic prototype cannot account for. Nonetheless, we can draw several insights from analyzing these scenarios and relate them back to the design considerations in the earlier chapters

13.10 Discussion

The above scenarios demonstrate the primary claim made throughout this report. Multi-cloud operations are as much about orchestration and mental overhead as they are about connecting to cloud APIs. With its ability to consolidate intent and reduce operational fragmentation, Nebula helps achieve better productivity, better governance, and better reproducibility. In particular, it allows startups to get their feet wet more quickly, helps enterprises establish governance standards, and enables researchers and academics to conduct their work in a more systematic fashion.

Table 13.1: Illustrative Cost and Fit Comparison by Scenario

Scenario	**Primary Goal**	**Suggested Cloud Bias**	**Nebula Value**
Student development sandbox	low cost	burstable mixed cloud	one workflow for repeat setup
Enterprise internal app	identity alignment	Azure-first	governance and inventory unification
Data-heavy analytics prototype	fast experimentation	GCP-first	compare supporting infrastructure centrally
Broad service integration platform	service breadth	AWS-first	consistent provisioning and failure handling
Disaster recovery standby	resilience	dual-provider	coordinated fallback orchestration

Appendix A

USER MANUAL: NEBULA COMMAND CENTER

This manual provides a step-by-step guide for setting up and using the Nebula Multi-Cloud platform. It includes separate sections for Windows, macOS, and Linux setup.

A.1 System Dependencies and OS-Specific Setup

A.1.1 Phase 1-A: Windows Setup (WSL2 Highly Recommended)

For Windows users, using the Windows Subsystem for Linux (WSL2) with Ubuntu 22.04 LTS is the most stable and performant environment for Nebula. 1) **Enable WSL2**: Open PowerShell as Administrator and run: `wsl --install`. 2) **Docker Desktop**: Install Docker Desktop and enable the WSL2 backend in the settings. 3) **Terraform**: Install using the Linux distribution's `apt-get` to ensure it matches the production worker environment.

A.1.2 Phase 1-B: macOS Setup (Homebrew Oriented)

macOS provides a robust Unix-based environment that is excellent for development. 1) **Install Homebrew**: `/bin/bash -c "`$(curl-fsSLhttps://raw.githubusercontent.com/Homebrew/install/$
$DownloadandinstalltheDMGforAppleSiliconorIntelasappropriate.3)$**`Terraform via Brew`**:
Run`brew install terraform`$togetthelatestbinary.$

A.1.3 Phase 1-C: Linux Setup (Native Deployment)

`Native Linux (Ubuntu 22.04+) is the preferred environment for the production Nebula worker cluster. 1)` **`Install Docker`**`: sudo apt-get install docker.io docker-compose. 2)` **`Terraform via Hashicorp Repo`**`: - Add the HashiCorp GPG key. - Add the HashiCorp repository. - sudo apt-get update  sudo apt-get`

install terraform. 3) **Python 3.11**: Ensure Python 3.11 and the venv module are installed for isolated virtual environments.

A.2 Local Backend Installation Guide

A.2.1 Phase 2: Environment Configuration

Create a .env file in the backend directory with the following variables:

```
DATABASE_URL=postgresql://user:pass@localhost:5432/nebula_db
SECRET_KEY=yoursymmetricsecretkeyhere
REDIS_URL=redis://localhost:6379/0
AI_COPILOT_API_KEY=youropenaikeyhere
```

A.2.2 Phase 3: Service Startup

Use the provided docker-compose.yml to start the peripheral services:

```
docker-compose up -d db redis
```

This will launch PostgreSQL and Redis in the background, allowing the FastAPI and Celery processes to connect immediately.

A.3 Cloud Provider Configuration

A.3.1 Registering AWS Credentials

1) Go to the AWS Console -> IAM -> Users. 2) Create a user with Programmatic Access and AdministratorAccess. 3) Copy the Access Key ID and Secret Access Key. 4) In Nebula, go to "Settings" -> "Add Provider" and select "AWS." 5) Paste your keys and click "Securely Register."

A.3.2 Registering Azure Credentials

Azure requires a Service Principal for programmatic access. 1) Open Azure CLI and run: az ad sp create-for-rbac --name Nebula. 2) Capture the appId, password, and tenantId. 3) In Nebula, go to "Add Provider" and select "Azure." 4) Enter the credentials to enable Azure VM/Storage provisioning.

A.3.3 Registering GCP Credentials

Google Cloud onboarding requires a service account with a JSON key and the correct project scope. 1) Open the Google Cloud Console and create or select

a project. 2) Navigate to IAM and Admin -> Service Accounts. 3) Create a service account with the minimum roles required for the resources you plan to manage. 4) Generate a JSON key and store it securely. 5) In Nebula, choose "Add Provider" -> "GCP" and upload or paste the service account fields. 6) Confirm the project ID and default region before saving the credential.

A.3.4 Environment Variable Reference

A.4 Using the Nebula Dashboard

A.4.1 Provisioning Your First Resource

1) Click the "+" icon in the top right corner. 2) Select your cloud provider (e.g., AWS). 3) Select "Virtual Machine" from the resource dropdown. 4) Enter a name (e.g., "my-dev-vm") and select "t3.micro." 5) Click "Launch Infrastructure" and watch the real-time logs in the "Cockpit" panel.

A.4.2 Recommended First-Day Validation

Before using live cloud accounts heavily, perform a short validation cycle:

1. create a test credential for one provider,
2. launch a low-cost resource in a sandbox region,
3. confirm that the task reaches the completed state,
4. verify that the resource appears in the inventory list,
5. trigger a manual sync and confirm metadata remains consistent,
6. delete the test resource and verify that cleanup completes successfully.

This sequence confirms that the core path of credential storage, background execution, Terraform generation, sync, and deletion is functioning correctly.

A.4.3 Daily Operator Workflow

An operator using Nebula in a lab or project environment can follow a simple day-to-day routine:

1. review the dashboard counters for pending or failed tasks,
2. check whether any provider credentials are expired or misconfigured,

3. launch or modify resources as needed for current work,
4. monitor logs during provisioning and investigate slow tasks,
5. run a synchronization cycle before generating reports or screenshots,
6. remove unused resources to avoid cost drift.

A.4.4 Cleanup Procedure

To clean up a project safely:

1. filter the resource table by project or tag,
2. delete non-critical compute resources first,
3. remove dependent storage or network objects only after confirming no tasks are active,
4. run a final sync to detect drift or residual assets,
5. archive logs if the environment is part of a graded evaluation or experiment.

A.5 Troubleshooting and FAQs

A.5.1 Common Issues and Remediation

- **Issue**: "PostgreSQL connection refused."
- **Solution**: Ensure the Docker container for the DB is running and the port mapping 5432:5432 is correct.
- **Issue**: "Terraform state locked."
- **Solution**: Delete the .terraform.tfstate.lock.info file in the workspace directory and retry.
- **Issue**: "Azure subscription not found."
- **Solution**: Verify the Tenant ID and ensure the Service Principal has Contributor permissions on the subscription.

Table A.1: Essential Environment Variables for Local Setup

Variable	Example	Purpose
DATABASE_URL	postgresql://user:pass@localhost:5432/nebula_db	database connection string
REDIS_URL	redis://localhost:6379/0	queue broker and task transport
SECRET_KEY	generated fernet key	encryption of provider secrets
AI_COPILOT_API_KEY	provider token	failure analysis integration
LOG_LEVEL	info	runtime verbosity tuning

Table A.2: Common Operational Issues and Recovery Actions

Symptom	Likely Cause	Recommended Action
Task remains queued	worker offline or broker unavailable	restart worker service and verify Redis connectivity
Logs stop midway	Terraform process stalled or provider timeout	inspect worker logs and retry with lower concurrency
Sync returns partial inventory	permission scope too narrow	validate provider role and region scope
AI analysis unavailable	missing API key or rate limit	verify key configuration and retry later
Repeated auth failures	expired credential or wrong secret format	re-register provider credential and test read access

A.5.2 Expanded Troubleshooting Matrix

A.5.3 Frequently Asked Questions

1. **Can Nebula manage multiple accounts of the same provider?** Yes. The credential vault can store multiple provider entries, provided each is tagged and scoped clearly.

2. **Should production credentials be used during development?** No. Use sandbox or low-privilege accounts whenever possible.

3. **Why does a task take longer than the API request?** Because the API only records intent and queues the work. The actual infrastructure execution happens asynchronously in the worker layer.

4. **What should I do if a resource is visible in the cloud console but missing in Nebula?** Trigger a manual synchronization cycle. If the resource still does not appear, verify whether the credential used by Nebula has permission to list that resource type.

Appendix B

CORE ENGINE CODE SNIPPETS

This appendix provides the architectural "blueprints" for the Nebula platform. Each snippet includes a technical description of its role in the multi-cloud orchestration engine.

B.1 Backend: The Provisioning Controller (FastAPI)

Listing B.1: Full Provisioning Router

```
@router.post("/resources/", response_model=ResourceSchema)
def create_resource(
    request: ResourceCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_active_user)
):
    # 1. Identity Verification
    if not current_user:
        raise HTTPException(status_code=401, detail="Unauthorized")

    # 2. Persist Intent (Transactional)
    new_resource = Resource(
        user_id=current_user.id,
        name=request.name,
        provider=request.provider,
        type=request.type,
        status="pending",
        configuration=request.variables
    )
    db.add(new_resource)
    db.commit()
    db.refresh(new_resource)

    # 3. Queue Asynchronous Execution
```

```
    provision_resource_task.delay(
        resource_id=new_resource.id,
        user_id=current_user.id,
        provider=request.provider,
        module=request.type,
        vars=request.variables
    )

    return new_resource
```

B.2 Security: The Encryption Service

Listing B.2: AES-128 Fernet Implementation

```
from cryptography.fernet import Fernet
import os

def get_vault_engine():
    key = os.getenv("SECRET_KEY")
    if not key:
        raise ValueError("SECRET_KEY␣not␣found␣in␣environment")
    return Fernet(key)

def encrypt_credential(plain_text: str) -> str:
    f = get_vault_engine()
    return f.encrypt(plain_text.encode()).decode()

def decrypt_credential(cipher_text: str) -> str:
    f = get_vault_engine()
    return f.decrypt(cipher_text.encode()).decode()
```

B.3 Backend: The Background Provisioning Worker

Listing B.3: Celery Worker Provisioning Flow

```
@celery.task(bind=True, max_retries=3, default_retry_delay=30)
def provision_resource_task(self, resource_id: int, user_id: int, provider
    : str, module: str, vars: dict):
    db = SessionLocal()
    resource = db.query(Resource).filter(Resource.id == resource_id).first
        ()

    try:
        resource.status = "running"
```

```
        db.commit()

        credential = load_credential_for_user(db, user_id, provider)
        secrets = decrypt_provider_bundle(credential)
        workspace = prepare_workspace(resource_id, provider, module, vars)

        write_provider_files(workspace, provider, module, vars)
        tf_env = build_terraform_env(secrets)

        init_result = run_command(["terraform", "init"], cwd=workspace,
            env=tf_env)
        if init_result.returncode != 0:
            raise RuntimeError(init_result.stderr)

        apply_result = run_command(
            ["terraform", "apply", "-auto-approve"],
            cwd=workspace,
            env=tf_env,
        )

        resource.execution_log = apply_result.stdout + "\n" + apply_result
            .stderr
        resource.status = "active" if apply_result.returncode == 0 else "
            failed"
        db.commit()

        if resource.status == "failed":
            analyze_with_ai_copilot.delay(resource.id)
    except RetryableCloudError as exc:
        resource.status = "retry_scheduled"
        db.commit()
        raise self.retry(exc=exc)
    except Exception as exc:
        resource.status = "failed"
        resource.execution_log = str(exc)
        db.commit()
        raise
    finally:
        db.close()
```

B.4 Backend: Inventory Synchronization Task

Listing B.4: Periodic Multi-Cloud Sync Task

```
@celery.task
```

```
def sync_all_resources():
    db = SessionLocal()
    credentials = db.query(CloudCredential).all()

    for credential in credentials:
        provider = credential.provider_type.lower()
        decrypted = decrypt_provider_bundle(credential)

        if provider == "aws":
            resources = fetch_aws_inventory(decrypted)
        elif provider == "azure":
            resources = fetch_azure_inventory(decrypted)
        elif provider == "gcp":
            resources = fetch_gcp_inventory(decrypted)
        else:
            continue

        for payload in resources:
            canonical = normalize_resource(provider, payload)
            upsert_resource_record(db, credential.user_id, canonical)

    db.commit()
    db.close()
```

B.5 Frontend: Resource Status Polling Hook

Listing B.5: React Hook for Task Polling

```
export const useTaskPolling = (taskId?: string) => {
  const [status, setStatus] = useState("idle");
  const [log, setLog] = useState("");

  useEffect(() => {
    if (!taskId) return;

    const timer = setInterval(async () => {
      const response = await axios.get(`/api/tasks/${taskId}`);
      setStatus(response.data.status);
      setLog(response.data.execution_log ?? "");

      if (["completed", "failed", "active"].includes(response.data.status)
          ) {
        clearInterval(timer);
      }
    }, 3000);
```

```
    return () => clearInterval(timer);
  }, [taskId]);

  return { status, log };
};
```

Appendix C

CONFIGURATION AND IAC SAMPLES

C.1 System Orchestration: Docker-Compose

This file defines the entire multi-container ecosystem of Nebula.

Listing C.1: docker-compose.yml with Worker Cluster

```
version: '3.8'

services:
  # 1. Peripheral Services
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: nebula_db
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  # 2. Application Core
  api:
    build: ./backend
    command: uvicorn app.main:app --host 0.0.0.0
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/nebula_db
      - SECRET_KEY=${SECRET_KEY}
    depends_on:
      - db
```

```
      - redis

  worker:
    build: ./backend
    command: celery -A app.tasks worker --loglevel=info
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/nebula_db
    depends_on:
      - redis
```

C.2 IaC Layer: AWS EC2 Terraform Sample

Listing C.2: AWS VM Module (main.tf)

```
resource "aws_instance" "nebula_vm" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name       = var.resource_name
    ManagedBy  = "Nebula-Orchestrator"
  }
}

variable "ami_id" { type = string }
variable "instance_type" { default = "t3.micro" }
variable "resource_name" { type = string }
```

C.3 IaC Layer: Azure Network Sample

Listing C.3: Azure VNet Module (network.tf)

```
resource "azurerm_resource_group" "nebula" {
  name     = var.resource_group_name
  location = var.location
}

resource "azurerm_virtual_network" "nebula_vnet" {
  name                = var.vnet_name
  address_space       = ["10.10.0.0/16"]
  location            = azurerm_resource_group.nebula.location
  resource_group_name = azurerm_resource_group.nebula.name
}

```

```
resource "azurerm_subnet" "nebula_subnet" {
  name                 = "default"
  resource_group_name  = azurerm_resource_group.nebula.name
  virtual_network_name = azurerm_virtual_network.nebula_vnet.name
  address_prefixes     = ["10.10.1.0/24"]
}
```

C.4 IaC Layer: GCP Compute Sample

Listing C.4: GCP VM Module (main.tf)

```
resource "google_compute_instance" "nebula_vm" {
  name         = var.instance_name
  machine_type = var.machine_type
  zone         = var.zone

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12"
    }
  }

  network_interface {
    network = "default"
    access_config {}
  }

  labels = {
    managed_by = "nebula"
    environment = var.environment
  }
}
```

C.5 Worker Queue Configuration

Listing C.5: Celery Queue Tuning Sample

```
broker_url = os.getenv("REDIS_URL", "redis://redis:6379/0")
result_backend = broker_url
task_serializer = "json"
result_serializer = "json"
accept_content = ["json"]
timezone = "UTC"
worker_prefetch_multiplier = 1
```

```
broker_transport_options = {
    "visibility_timeout": 3600,
}
```

Appendix D

FULL HCL TEMPLATES

This appendix provides the complete Terraform HCL (HashiCorp Configuration Language) templates used by the Nebula engine for resource provisioning across AWS, Azure, and GCP.

D.1 AWS VPC and Subnet Configuration

Listing D.1: AWS Infrastructure (vpc.tf)

```
resource "aws_vpc" "nebula_main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "nebula-vpc"
  }
}

resource "aws_subnet" "nebula_public" {
  vpc_id                  = aws_vpc.nebula_main.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true

  tags = {
    Name = "nebula-public-subnet"
  }
}
```

D.2 Azure Resource Group and Network Configuration

Listing D.2: Azure Infrastructure (network.tf)

```
resource "azurerm_resource_group" "nebula_rg" {
  name     = "nebula-resources"
  location = "East␣US"
}

resource "azurerm_virtual_network" "nebula_vnet" {
  name                = "nebula-vnet"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.nebula_rg.location
  resource_group_name = azurerm_resource_group.nebula_rg.name
}
```

D.3 GCP Compute and Network Configuration

Listing D.3: GCP Infrastructure (main.tf)

```
resource "google_compute_network" "nebula_network" {
  name                    = "nebula-network"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "nebula_subnet" {
  name          = "nebula-subnet"
  ip_cidr_range = "10.20.1.0/24"
  region        = "asia-south1"
  network       = google_compute_network.nebula_network.id
}

resource "google_compute_firewall" "allow_ssh" {
  name    = "nebula-allow-ssh"
  network = google_compute_network.nebula_network.name

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }

  source_ranges = ["0.0.0.0/0"]
}
```

Appendix E

DETAILED API DOCUMENTATION

This appendix provides a comprehensive reference for all REST API endpoints exposed by the Nebula backend.

E.1 Sample Request: Provision a Resource

Listing E.1: Example Provisioning Request Payload

```
POST /resources/provision
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "name": "demo-web-vm",
  "provider": "AWS",
  "resource_type": "virtual_machine",
  "region": "ap-south-1",
  "variables": {
    "instance_type": "t3.micro",
    "ami_id": "ami-xxxxxxxx",
    "tags": {
      "project": "blackbook-demo",
      "managed_by": "nebula"
    }
  }
}
```

E.2 Sample Response: Accepted Task

Listing E.2: Provisioning Response Payload

```
{
```

```
  "id": 42,
  "task_id": "c7d6d8aa-11b0-4b93-9cb5-001122334455",
  "status": "pending",
  "message": "Provisioning job accepted and queued"
}
```

E.3 Sample Response: AI Log Analysis

Listing E.3: AI Copilot Analysis Response

```
{
  "task_id": "c7d6d8aa-11b0-4b93-9cb5-001122334455",
  "category": "quota",
  "summary": "The target region does not have enough vCPU quota for the
     requested size.",
  "remedy": "Request a quota increase or redeploy with a smaller instance
     size.",
  "confidence": 0.88,
  "auto_fix": false
}
```

Table E.1: Nebula API Endpoint Reference

Ver	**Endpoint**	**Auth**	**Description**
POST	/auth/login	None	Authenticates user and returns JWT.
GET	/resources/	JWT	Fetches all resources for the logged-in user.
POST	/resources/provision	JWT	Queues a new provisioning task in Redis.
GET	/resources/id	JWT	Fetches detailed metadata for a specific resource ID.
DELETE	/resources/id	JWT	Queues a deletion task for the specified resource.
GET	/sync/all	JWT	Triggers a background sync task across all cloud providers.
GET	/logs/task_id	JWT	Retrieves the execution log for a specific task.
POST	/ai/analyze	JWT	Sends logs to the LLM agent for failure analysis.

Appendix F

DATABASE DDL STATEMENTS

This appendix provides the SQL (PostgreSQL) DDL (Data Definition Language) statements used to initialize the platform's persistence layer.

F.1 Table: Users

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    hashed_password TEXT NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

F.2 Table: Cloud Credentials (Encrypted)

```
CREATE TABLE cloud_credentials (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    provider_type VARCHAR(20) NOT NULL,
    access_key_enc TEXT NOT NULL,
    secret_key_enc TEXT NOT NULL,
    tenant_id_enc TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

F.3 Table: Resources

```
CREATE TABLE resources (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    provider VARCHAR(20) NOT NULL,
    resource_type VARCHAR(50) NOT NULL,
    status VARCHAR(20) DEFAULT 'pending',
    provider_id VARCHAR(100),
    metadata JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

F.4 Table: Tasks

```
CREATE TABLE tasks (
    id SERIAL PRIMARY KEY,
    resource_id INTEGER REFERENCES resources(id),
    task_uuid VARCHAR(100) UNIQUE NOT NULL,
    status VARCHAR(30) NOT NULL,
    execution_log TEXT,
    started_at TIMESTAMP,
    completed_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

F.5 Table: Activity Logs

```
CREATE TABLE activity_logs (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    action VARCHAR(80) NOT NULL,
    provider VARCHAR(20),
    target_id VARCHAR(120),
    details JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

F.6 Indexes and Query Optimization Aids

```
CREATE INDEX idx_resources_user_provider
    ON resources(user_id, provider);

```

```sql
CREATE INDEX idx_tasks_status
    ON tasks(status);

CREATE INDEX idx_activity_logs_user_created
    ON activity_logs(user_id, created_at DESC);
```

Appendix G

API DATA SCHEMAS

This appendix provides the formal Pydantic (Python) models and TypeScript (React) interfaces used for data validation across the platform.

G.1 Generic Resource Schema (Python)

```python
class ResourceBase(BaseModel):
    name: str = Field(..., max_length=100)
    provider: ProviderEnum
    resource_type: str
    region: Optional[str] = "us-east-1"
    variables: dict = {}

class ResourceCreate(ResourceBase):
    pass

class ResourceSchema(ResourceBase):
    id: int
    status: str
    provider_id: Optional[str]
    created_at: datetime

    class Config:
        from_attributes = True
```

G.2 Security Schema (TypeScript)

```typescript
interface AuthToken {
    access_token: string;
    token_type: string;
```

```
}

interface UserProfile {
    id: number;
    username: string;
    email: string;
    is_active: boolean;
}
```

G.3 Task and Analysis Schemas (Python)

```
class TaskSummary(BaseModel):
    task_id: str
    resource_id: int
    status: str
    started_at: Optional[datetime]
    completed_at: Optional[datetime]

class AIAnalysisSchema(BaseModel):
    task_id: str
    category: str
    summary: str
    remedy: str
    confidence: float = Field(..., ge=0.0, le=1.0)
    auto_fix: bool = False
```

G.4 Frontend Resource Types (TypeScript)

```
interface CloudResource {
    id: number;
    name: string;
    provider: "AWS" | "Azure" | "GCP";
    resource_type: string;
    region: string;
    status: "pending" | "running" | "active" | "failed";
    provider_id?: string;
}

interface TaskStatusResponse {
    task_id: string;
    status: string;
    execution_log?: string;
    analysis?: {
```

```
        category: string;
        summary: string;
        remedy: string;
        confidence: number;
    };
}
```

Appendix H

PROJECT GLOSSARY

H.1 Cloud Terms

- **AWS (Amazon Web Services)**: A subsidiary of Amazon providing on-demand cloud computing platforms and APIs.
- **Azure**: A cloud computing service created by Microsoft for building, testing, deploying, and managing applications.
- **GCP (Google Cloud Platform)**: A suite of cloud computing services that runs on the same infrastructure that Google uses internally.
- **Multi-Cloud**: The use of multiple cloud computing and storage services in a single heterogeneous architecture.
- **Vendor Lock-in**: A situation in which a customer is dependent on a vendor for products and services and cannot switch to another vendor without substantial costs.

H.2 Technical Terms

- **FastAPI**: A modern, high-performance web framework for building APIs with Python 3.7+ based on standard Python type hints.
- **Terraform**: An open-source infrastructure as code software tool created by HashiCorp.
- **Celery**: An asynchronous task queue/job queue based on distributed message passing.
- **Redis**: An in-memory data structure store used as a distributed, in-memory key-value database, cache, and message broker.

- **JWT (JSON Web Token)**: An open standard that defines a compact and self-contai way for securely transmitting information between parties as a JSON object.
- **AES-128**: Advanced Encryption Standard with a 128-bit key length, used for securing cloud credentials in the Nebula vault.

H.3 Operational Terms

- **Inventory Drift**: The mismatch between the state stored in Nebula and the actual state in a cloud provider.
- **Control Plane**: The logical layer responsible for coordination, automation, and management decisions.
- **Worker**: The asynchronous execution process that performs long-running cloud actions.
- **Canonical Model**: A normalized internal representation used to hide provider-s payload differences.
- **Runbook**: A documented set of actions used to respond to an operational issue.

H.4 Acronyms Used in the Blackbook

Table H.1: Common Acronyms and Their Meanings

Acronym	**Expanded Form**
API	Application Programming Interface
IaC	Infrastructure as Code
JWT	JSON Web Token
VPC	Virtual Private Cloud
VNet	Virtual Network
DFD	Data Flow Diagram
RTM	Requirement Traceability Matrix
SLO	Service Level Objective
DR	Disaster Recovery

Appendix I

DATA FLOW DIAGRAMS (DFD)

This appendix provides a detailed narrative and structural description of the data flow across the Nebula platform, from the initial user request to the final cloud state persistence.

I.1 DFD Level 0: Global Context Diagram

1) **Entities**: User Browser, Nebula Control Plane, Cloud Provider APIs (AWS/Azure/GC
2) **Primary Flow**: The user sends "Infrastructure Intent" to the Control Plane. The Control Plane returns "Consolidated Inventory" and "Execution Logs." 3) **Cloud Interaction**: The Control Plane sends "Authenticated API Requests" to the providers and receives "Resource Metadata."

I.2 DFD Level 1: Service Decomposition

The Level 1 DFD breaks down the Control Plane into its core services: API Gateway, Task Broker, and worker Cluster. 1) **Flow 1 (Auth)**: User -> API (Verificat -> Database. 2) **Flow 2 (Provisioning)**: User -> API -> Redis (Task Queue) -> Worker (Terraform). 3) **Flow 3 (Logging)**: Worker -> Database (Log Storage) -> API (WebSocket) -> User UI.

I.3 DFD Level 2: Task Runner Internal Flow

This level focuses on the internal state transitions of the Celery Worker during a Terraform execution. - **Stage A**: Receive Task ID and Vault Key. - **Stage B**: Fetch Encrypted Credentials. - **Stage C**: Generate HCL Template. - **Stage D**: Initialize Terraform Backend. - **Stage E**: Execute 'Apply' and capture STDOUT/STDERR. - **Stage F**: Update Database and mark Task as 'Complete'.

[node distance=4.0cm, auto, scale=0.7, every node/.style=scale=0.7] entity = [rectangle, draw, rounded corners, fill=blue!8, text centered, minimum width=8em, minimum height=3em] process = [rectangle, draw, rounded corners, fill=green!10, text centered, minimum width=10em, minimum height=3.4em] line = [draw, -latex']
[entity] (user) User Browser; [process, right=of user] (nebula) Nebula Control Plane; [entity, right=of nebula] (cloud) AWS / Azure / GCP APIs;
[line] (user) – node[above] Infrastructure Intent (nebula); [line] (nebula) – node[below] Inventory, Logs, Status (user); [line] (nebula) – node[above] Authenticated API Requests (cloud); [line] (cloud) – node[below] Resource Metadata (nebula);

Figure I.1: DFD Level 0 for the Nebula Multi-Cloud Platform

[node distance=3.4cm, auto, scale=0.48, every node/.style=scale=0.48] service = [rectangle, draw, rounded corners, fill=orange!10, text centered, minimum width=8em, minimum height=3em] store = [rectangle, draw, fill=gray!12, text centered, minimum width=7em, minimum height=2.8em] line = [draw, -latex']
[service] (user) User Browser; [service, right=of user] (api) API Gateway; [store, below=of api] (db) PostgreSQL; [store, right=of api] (redis) Redis Broker; [service, below=of redis] (worker) Worker Cluster; [service, right=of redis] (cloud) Cloud APIs;
[line] (user) – node[above] REST / UI Actions (api); [line] (api) – node[left] Auth / State (db); [line] (api) – node[above] Queue Task (redis); [line] (redis) – node[right] Dispatch (worker); [line] (worker) – node[above] Terraform / SDK Calls (cloud); [line] (cloud) – node[below] Provider Responses (worker); [line] (worker) – node[below] Logs / Resource State (db); [line] (api) – node[below] Inventory / Logs (user);

Figure I.2: DFD Level 1 Showing Service Decomposition Inside Nebula

[node distance=1.35cm, auto, scale=0.85, every node/.style=scale=0.85] step = [rectangle, draw, rounded corners, fill=purple!10, text centered, minimum width=13em, minimum height=2.8em] line = [draw, -latex']
[step] (a) Receive Task ID; [step, below=of a] (b) Fetch and Decrypt Credentials; [step, below=of b] (c) Generate Terraform HCL; [step, below=of c] (d) Run terraform init; [step, below=of d] (e) Run terraform apply / capture logs; [step, below=of e] (f) Update Database and Final State;
[line] (a) – (b); [line] (b) – (c); [line] (c) – (d); [line] (d) – (e); [line] (e) – (f);

Figure I.3: DFD Level 2 for Internal Task Runner Execution

I.4 DFD Level 3: AI Copilot Analysis Flow

The Level 3 DFD zooms into the failure-analysis sub-flow:

1. Worker stores raw execution log after a failed task.
2. API retrieves the failed task metadata and strips secrets or sensitive tokens.
3. Sanitized log excerpt is combined with provider, region, and resource context.
4. AI copilot receives the bounded prompt and returns a structured explanation.
5. The explanation is stored and made visible to the user in the dashboard.

This decomposition is important because it highlights the separation between raw operational data, sanitized analysis input, and user-facing remediation output.

I.5 Data Stores and Trust Boundaries

Across all DFD levels, the following trust boundaries should be recognized:

- **Browser to API**: authenticated but untrusted client boundary.
- **API to Database**: trusted application boundary for persistent state.
- **API to Broker**: asynchronous handoff boundary.
- **Worker to Cloud APIs**: privileged execution boundary using decrypted provider credentials.
- **Worker to AI Analyzer**: sanitized diagnostic boundary without raw secrets.

Mapping these trust boundaries helps explain why the architecture isolates credential handling and why logs must be filtered before external analysis.

[node distance=1.8cm, auto, scale=0.82, every node/.style=scale=0.82] block = [rectangle, draw, rounded corners, fill=teal!10, text centered, minimum width=8em, minimum height=3em] line = [draw, -latex’]
[block] (failed) Failed Task Log; [block, right=of failed] (sanitize) Sanitization Layer; [block, right=of sanitize] (context) Context Builder; [block, right=of context] (copilot) AI Copilot; [block, below=of context] (store) Analysis Store; [block, left=of store] (ui) User Dashboard;
[line] (failed) – (sanitize); [line] (sanitize) – (context); [line] (context) – (copilot); [line] (copilot) – (store); [line] (store) – (ui); [line, dashed] (ui.north) |- (failed.south);

Figure I.4: DFD Level 3 for the Nebula AI Failure-Analysis Subsystem

Appendix J

DETAILED TEST REPORTS AND BUG TRACKING

This appendix provides a comprehensive record of the testing phase, including 100+ test cases (TC-01 through TC-100) and a historical log of solved bugs.

J.1 Exhaustive Test Case Matrix (TC-01 to TC-100)

J.2 Test Execution Protocol

The testing process followed a layered order so that failures could be localized quickly:

1. validate schemas and input constraints at the unit level,
2. test API routes with mocked database sessions,
3. verify queue delivery and worker execution using integration tests,
4. execute sandbox cloud tasks with low-cost resources,
5. review dashboard behavior and log presentation from an operator perspective.

J.3 Representative Detailed Test Extract

Listing J.1: Detailed Test Extract

```
TC-01 | Auth     | Login with valid credentials                    | PASS |
    JWT issued
TC-02 | Auth     | Login with invalid password                     | PASS |
    401 returned
```

```
TC-03 | Vault    | Encrypt AWS access key                          | PASS |
    ciphertext stored
TC-04 | Vault    | Decrypt credential during worker execution     | PASS |
    plaintext not logged
TC-05 | API      | Reject malformed resource payload              | PASS |
    schema validation hit
TC-06 | Queue    | Enqueue provisioning task                      | PASS |
    broker accepted message
TC-07 | Queue    | Worker picks queued task                       | PASS |
    status moved to running
TC-08 | IaC      | Terraform init for AWS module                  | PASS |
    providers resolved
TC-09 | IaC      | Terraform apply for sample VM                  | PASS |
    resource active
TC-10 | Sync     | Refresh AWS inventory                          | PASS |
    canonical records updated
TC-11 | Sync     | Refresh Azure inventory                        | PASS |
    canonical records updated
TC-12 | Sync     | Refresh GCP inventory                          | PASS |
    canonical records updated
TC-13 | AI       | Detect AccessDenied signature                  | PASS |
    mapped to IAM category
TC-14 | AI       | Detect quota exhaustion                        | PASS |
    quota remediation shown
TC-15 | UI       | Show running status badge                      | PASS |
    badge transitions correct
TC-16 | UI       | Stop polling after task completion             | PASS |
    timer cleared
TC-17 | Audit    | Log delete action                              | PASS |
    activity row written
TC-18 | Recovery| Retry transient cloud timeout                   | PASS |
    next retry scheduled
```

J.4 Historical Bug Log and Resolutions

The following log tracks critical bugs identified during the integration testing phase and their subsequent fixes.

- **Bug ID: B-102 (Credential Scrambling)**
  - *Issue*: Special characters in Azure secrets were causing Fernet decryption errors.
  - *Fix*: Implemented Base64 encoding/decoding before encryption/decryption to ensure character integrity.

- **Bug ID: B-205 (Redis Queue Congestion)**
  - *Issue*: Bulk provisioning of 20+ VMs caused the Redis broker to timeout.
  - *Fix*: Increased the visibility_timeout in Celery and optimized the database connection pool in the worker cluster.
- **Bug ID: B-309 (UI State Mismatch)**
  - *Issue*: Dashboard counts did not update automatically after a resource was deleted from the cloud portal directly.
  - *Fix*: Implemented a periodic "background reconcile" task that triggers every 5 minutes to keep the database in sync with actual cloud state.
- **Bug ID: B-411 (Task Retry Storm)**
  - *Issue*: Retryable provider errors caused too many workers to retry in parallel, amplifying queue depth.
  - *Fix*: Added jitter to the exponential backoff calculation and reduced worker prefetch.
- **Bug ID: B-507 (Unsafe Log Disclosure)**
  - *Issue*: Raw Terraform error output could include values that should not be forwarded to the AI analyzer.
  - *Fix*: Added sanitization rules for secrets, access keys, endpoint tokens, and private IP patterns before analysis.

Table J.1: Nebula 100-Item Test Matrix (Summary)

Test ID	Category	Description	Pass/Fail	Notes
TC-31	UI	Dashboard Sidebar Collapse	PASS	-
TC-45	Security	Brute Force Protection	PASS	5 attempts limit
TC-52	API	Invalid Resource Name Check	PASS	Returns 400 Bad Req
TC-68	Sync	AWS Multi-Region Discovery	PASS	Verified us-east-1/2
TC-75	Vault	Key Rotation Logic	PASS	Atomic update
TC-88	Network	VNet Peering Success	PASS	Azure backbone confirmed
TC-99	Final	24hr System Stability	PASS	0 Memory leaks

Appendix K

DOCUMENTATION DIAGRAMS IMPORTED FROM docs/

This appendix consolidates the visual diagrams already present in the project's docs/ directory and makes them part of the formal blackbook. The blocks below are adapted from the source documentation files so that the report preserves the same architectural views, flow descriptions, and structural summaries used by the engineering documentation.

K.0.1 Component Interaction Diagram

Source: docs/README.md

Listing K.1: Component Interaction Diagram Adapted from docs/README.md

```
+--------------------------- FRONTEND ------------------------------+
| +-----------+   +-------------+   +-------------+                 |
| | Dashboard  |-->| React Query |-->| Axios API   |                 |
| | Component  |   |   Cache     |   |   Client    |                 |
| +-----------+   +-------------+   +------+------+                 |
+----------------------------------------------|----------------------+
                                               | HTTP/JSON
                                               v
+--------------------------- BACKEND -------------------------------+
| +-------------------------------------------------------------+   |
| | FastAPI Application                                         |   |
| | +-----------+ +-----------+ +-------------+                 |   |
| | |Middleware | | Routers   | |Dependencies |                 |   |
| | | CORS      | | /auth     | | get_db()    |                 |   |
| | | Auth      | | /dashboard| | get_user()  |                 |   |
| | | Logging   | | /inventory| |             |                 |   |
| | +-----------+ +-----------+ +-------------+                 |   |
| | Endpoint handlers query and prepare response payloads       |   |
| +-----------------------------+-------------------------------+   |
|                               |                                   |
|                               v                                   |
|                    +---------------------------+                  |
|                    | SQLAlchemy ORM            |                  |
|                    | session.query(...).all()  |                  |
|                    +-------------+-------------+                  |
+-----------------------------------|-------------------------------+
                                    v
+--------------------------- POSTGRESQL ------------------------------+
| tables: users, credentials, resource_inventory, cost_data,         |
| provider_health, terraform_states, resources                       |
+---------------------------------------------------------------------+
```

```
CELERY WORKER INTERACTION: Celery Beat --> Redis Queue --> Celery Worker --> PostgreSQL
```

K.1 Diagrams from the Repository

K.1.1 High-Level Architecture

Source: `docs/README.md`

Listing K.2: High-Level Architecture Adapted from docs/README.md

```
+-------------------------------------------------------------------+
|                        CLOUD PROVIDERS                            |
|                                                                   |
|   +------------+    +------------+    +------------+             |
|   |    AWS     |    |   Azure    |    |    GCP     |             |
|   | - EC2      |    | - VMs      |    | - Compute  |             |
|   | - S3       |    | - Storage  |    | - Storage  |             |
|   | - VPC      |    | - VNet     |    | - VPC      |             |
|   | - Cost API |    | - Cost Mgmt|    | - Billing  |             |
|   +-----+------+    +-----+------+    +-----+------+             |
+---------+------------------+------------------+-------------------+
          |                  |                  |
          v                  v                  v
+-------------------------------------------------------------------+
|                    BACKGROUND SYNC LAYER                          |
|                                                                   |
|   +-----------------------------------------------------------+   |
|   | Celery Beat Scheduler (triggers sync every 10 minutes)    |   |
|   +---------------------------+-------------------------------+   |
|                               |                                   |
|   +---------------------------v-------------------------------+   |
|   | Celery Workers                                            |   |
|   | - AWS Sync Service    - Azure Sync Service                |   |
|   | - GCP Sync Service    - Terraform Provisioning Tasks      |   |
|   +---------------------------+-------------------------------+   |
+-------------------------------+-----------------------------------+
                                |
                                v
+-------------------------------------------------------------------+
|                          DATA LAYER                               |
|                                                                   |
|   +-----------------------------------------------------------+   |
|   | PostgreSQL: users, credentials, inventory, costs, health  |   |
|   | Terraform states, deployment history                      |   |
|   +-----------------------------------------------------------+   |
|   +-----------------------------------------------------------+   |
|   | Redis: task queue, session storage, cache                 |   |
|   +-----------------------------------------------------------+   |
+-------------------------------+-----------------------------------+
                                |
                                v
+-------------------------------------------------------------------+
|                           API LAYER                               |
|                                                                   |
|   +-----------------------------------------------------------+   |
|   | FastAPI Backend                                           |   |
|   | - /auth    - /dashboard    - /inventory                   |   |
|   | - /resources - /billing    - /credentials                 |   |
|   | - JWT auth, request validation, CORS, error handling      |   |
|   +-----------------------------------------------------------+   |
+-------------------------------+-----------------------------------+
                                |
                                v
+-------------------------------------------------------------------+
|                      PRESENTATION LAYER                           |
|                                                                   |
|   +-----------------------------------------------------------+   |
|   | React Frontend: Dashboard, VMs, Storage, Deployments,     |   |
|   | Costs, real-time updates, live logs, charts               |   |
|   +-----------------------------------------------------------+   |
+-------------------------------------------------------------------+
```

K.1.2 Data Flow Diagram

Source: `docs/README.md`

Listing K.3: Data Flow Diagram Adapted from docs/README.md

```
+-----------+       1. HTTP Request        +----------------------+
|   User    | --------------------------> |    FastAPI Backend   |
| (Browser) |                              | 2. Authenticate JWT  |
+-----+-----+                              | 3. Validate request  |
      ^                                    | 4. Query database    |
      | 10. JSON Response                  +----------+-----------+
      |                                               |
      |                                               | 5. SQL Query
      |                                               v
+-----+-----+                               +----------------------+
|  React UI  | <-------------------------- |     PostgreSQL       |
| updates UI | 9. HTTP response formatted   | 6. Fetch cached data |
+-----------+                               +----------+-----------+
                                                       |
                                                       | 7. Return rows
                                                       v
                                             +----------------------+
                                             |    FastAPI Backend   |
                                             | 8. Format JSON       |
                                             +----------------------+

BACKGROUND SYNC FLOW

Celery Beat Scheduler
        |
        | Enqueue task
        v
Celery Worker
        |
        | Call cloud APIs
        v
Cloud Provider APIs
        |
        | Return resources
        v
Celery Worker normalizes data
        |
        | INSERT / UPDATE
        v
PostgreSQL resource_inventory
```

K.1.3 Dashboard Request Flow Example

Source: `docs/README.md`

Listing K.4: Dashboard Request Flow Adapted from docs/README.md

```
1. User opens browser -> http://localhost:5173
2. React app loads -> Dashboard component mounts
3. useQuery hook triggers -> api.get('/dashboard/stats')
4. Axios sends GET /dashboard/stats with JWT token
5. FastAPI verifies JWT, extracts user
6. Route handler queries ResourceInventory and CostData
7. SQLAlchemy executes SQL against PostgreSQL
8. PostgreSQL returns results
9. FastAPI formats JSON response
10. Response sent to frontend
11. React Query caches data
12. Dashboard re-renders with real metrics and charts
```

K.2 Diagrams for BACKEND

K.2.1 Backend Project Structure

Source: `docs/BACKEND.md`

Listing K.5: Backend Project Structure Adapted from docs/BACKEND.md

```
backend/
|-- app/
|   |-- __init__.py
|   |-- main.py                 # FastAPI app entry point
|   |-- worker.py               # Celery worker entry point
|   |
|   |-- api/
|   |   |-- __init__.py
|   |   `-- endpoints/
|   |       |-- auth.py
|   |       |-- dashboard.py
|   |       |-- inventory.py
|   |       |-- billing.py
|   |       |-- resources.py
|   |       `-- credentials.py
|   |
|   |-- core/
|   |   |-- __init__.py
|   |   |-- config.py
|   |   |-- security.py
|   |   `-- celery_app.py
|   |
|   |-- db/
|   |   |-- __init__.py
|   |   |-- base.py
|   |   `-- migrate.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- user.py
|   |   |-- credential.py
|   |   |-- resource.py
|   |   `-- resource_inventory.py
|   |
|   |-- schemas/
|   |   |-- __init__.py
|   |   |-- user.py
|   |   |-- resource.py
|   |   `-- dashboard.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   |-- aws_sync.py
|   |   |-- azure_sync.py
|   |   |-- gcp_sync.py
|   |   |-- terraform_runner.py
|   |   `-- cloud_sync.py
|   |
|   `-- tasks/
|       |-- __init__.py
|       |-- sync_tasks.py
|       `-- terraform_tasks.py
|
|-- requirements.txt
|-- Dockerfile
`-- .env.example
```

K.3 Diagrams for DATABASE

K.3.1 Database Entity Relationship Diagram

Source: `docs/DATABASE.md`

Listing K.6: Database ER Diagram Adapted from docs/DATABASE.md

```
+-------------+
|   users     |
|-------------|
| id (PK)     |
| email       |<-------------------------------+
| password    |                                |
| full_name   |                                |
| created_at  |                                |
+-------------+                                |
       | user_id (FK)                          |
       +---------------+---------------+-------+--------------------+
                       |               |                            |
                       v               v                            v
             +----------------+  +---------------+       +----------------+
             | credentials    |  | resources     |       | cost_data      |
             |----------------|  |---------------|       |----------------|
             | id (PK)        |  | id (PK)       |       | id (PK)        |
             | user_id (FK)   |  | user_id (FK)  |       | user_id (FK)   |
             | provider       |  | name          |       | provider       |
             | name           |  | provider      |       | service        |
             | access_key(enc)|  | type          |       | resource_id    |
             | secret_key(enc)|  | status        |       | cost           |
             | region         |  | details       |       | currency       |
             | is_active      |  | tf_output     |       | date           |
             | created_at     |  | created_at    |       | billing_period |
             +----------------+  | updated_at    |       | metadata       |
                                 +-------+-------+       +----------------+
                                         |
                                         | resource_id (FK)
                                         v
                                 +-------------------+
                                 | terraform_states  |
                                 |-------------------|
                                 | id (PK)           |
                                 | resource_id (FK)  |
                                 | state_data (JSON) |
                                 | version           |
                                 | created_at        |
                                 +-------------------+

Additional user-owned operational tables:
- resource_inventory(user_id FK)
- provider_health(user_id FK)
```

K.4 Diagrams for CELERY

K.4.1 Celery Architecture

Source: `docs/CELERY.md`

Listing K.7: Celery Architecture Adapted from docs/CELERY.md

```
+-------------------------------------------------------------+
|                      CELERY BEAT                            |
|                     (Scheduler)                             |
| Periodic tasks:                                             |
| - sync_all_users_resources() every 10 min                   |
| - cleanup_old_logs() daily at midnight                      |
| - update_provider_health() every 5 min                      |
+-------------------------+-----------------------------------+
                          |
                          | Enqueue tasks
                          v
+-------------------------------------------------------------+
|                        REDIS                                |
|                  (Message Broker)                           |
| Queues:                                                     |
| - celery              - celery:sync                         |
| - celery:terraform    - celery:results                      |
+-------------------------+-----------------------------------+
                          |
                          | Workers dequeue tasks
                          v
+-------------------------------------------------------------+
|                     CELERY WORKERS                          |
| Worker pool:                                                |
| +----------+  +----------+  +----------+                   |
| | Worker 1 |  | Worker 2 |  | Worker 3 |                   |
| | AWS Sync |  |Azure Sync|  | Idle     |                   |
| +----------+  +----------+  +----------+                   |
| Tasks:                                                      |
| - sync_aws_resources()                                      |
| - sync_azure_resources()                                    |
| - sync_gcp_resources()                                      |
| - provision_resource_task()                                 |
| - destroy_resource_task()                                   |
+-------------------------+-----------------------------------+
                          |
                          | Read / write data
                          v
+-------------------------------------------------------------+
|                       POSTGRESQL                            |
| - resource_inventory                                        |
| - cost_data                                                 |
| - provider_health                                           |
| - resources                                                 |
+-------------------------------------------------------------+
```

K.4.2 Security Layers

Source: `docs/SECURITY.md`

Listing K.8: Security Layers Adapted from docs/SECURITY.md

```
+-------------------------------------------------------+
|                    USER LAYER                         |
| - JWT Authentication                                  |
| - Password hashing (bcrypt)                           |
| - Session management                                  |
+-------------------------+-----------------------------+
                          |
                          v
+-------------------------------------------------------+
|                     API LAYER                         |
| - HTTPS / TLS encryption                              |
| - CORS protection                                     |
| - Rate limiting                                       |
| - Input validation (Pydantic)                         |
+-------------------------+-----------------------------+
                          |
                          v
+-------------------------------------------------------+
|                 APPLICATION LAYER                     |
| - Credential encryption (AES-256)                     |
| - Environment variable management                     |
| - Secret key rotation                                 |
+-------------------------+-----------------------------+
                          |
                          v
+-------------------------------------------------------+
|                   DATABASE LAYER                      |
| - Encrypted connections (SSL)                         |
| - User isolation                                      |
| - Encrypted credentials at rest                       |
+-------------------------------------------------------+
```

K.4.3 Encryption and Decryption Flow

Source: `docs/SECURITY.md`

Listing K.9: Encryption and Decryption Flow Adapted from docs/SECURITY.md

```
Encryption:
Plain Credential -> AES-256 Encryption -> Encrypted Blob -> Database
                    (with SECRET_KEY)

Decryption:
Database -> Encrypted Blob -> AES-256 Decryption -> Plain Credential
                                  (with SECRET_KEY)
```

K.4.4 High-Level Components

Source: `docs/SUMMARY.md`

Listing K.10: High-Level Components Adapted from docs/SUMMARY.md

```
+-------------------------------------------------------+
|                  CLOUD PROVIDERS                      |
|                   AWS | Azure | GCP                   |
+-------------------------+-----------------------------+
                          |
                          v
+-------------------------------------------------------+
|               BACKGROUND SYNC LAYER                    |
|           Celery Workers + Beat Scheduler              |
|         (every 10 min: sync all resources)             |
```

```
+-------------------------+-------------------------------+
                          |
                          v
+-----------------------------------------------------------+
|                       DATA LAYER                          |
|        PostgreSQL (resources, costs, health)              |
|        Redis (task queue, cache)                          |
+-------------------------+-------------------------------+
                          |
                          v
+-----------------------------------------------------------+
|                        API LAYER                          |
|                     FastAPI Backend                       |
|      (Auth, Dashboard, Inventory, Billing, Resources)     |
+-------------------------+-------------------------------+
                          |
                          v
+-----------------------------------------------------------+
|                 PRESENTATION LAYER                        |
|                    React Frontend                         |
|      (Dashboard, VMs, Storage, Deployments, Costs)        |
+-----------------------------------------------------------+
```

K.4.5 Relationship Summary

Source: `docs/SUMMARY.md`

Listing K.11: Relationship Summary Adapted from docs/SUMMARY.md

```
users (1) ---- (N) credentials
      (1) ---- (N) resource_inventory
      (1) ---- (N) cost_data
      (1) ---- (N) provider_health
      (1) ---- (N) resources

resources (1) ---- (N) terraform_states
```

K.4.6 Task Flow

Source: `docs/SUMMARY.md`

Listing K.12: Task Flow Adapted from docs/SUMMARY.md

```
Celery Beat (every 10 min)
    |
    v
sync_all_users_resources()
    |
    v
For each user:
    sync_user_resources(user_id)
        |
        v
    For each credential:
        sync_aws_resources() OR
        sync_azure_resources() OR
        sync_gcp_resources()
            |
            v
        Fetch from cloud API
            |
            v
        Upsert to database
            |
            v
        Update provider health
```

K.4.7 User Views Dashboard Flow

Source: `docs/SUMMARY.md`

Listing K.13: User Dashboard Flow Adapted from docs/SUMMARY.md

```
1. User opens browser -> http://localhost:5173
2. React app loads Dashboard component
3. useQuery hook calls api.get('/dashboard/stats')
4. Axios sends GET /dashboard/stats with JWT token
5. FastAPI verifies JWT and extracts user
6. Query database for inventory and billing statistics
7. Format JSON response
8. Send data to frontend
9. React Query caches response
10. Dashboard renders with real data
```

K.4.8 Background Resource Sync Flow

Source: `docs/SUMMARY.md`

Listing K.14: Background Resource Sync Flow Adapted from docs/SUMMARY.md

```
1. Celery Beat triggers every 10 minutes
2. sync_all_users_resources() starts
3. Query all users with credentials
4. For each user enqueue sync_user_resources(user_id)
5. For each credential:
   - If AWS: sync_aws_resources()
   - If Azure: sync_azure_resources()
   - If GCP: sync_gcp_resources()
6. Sync task executes:
   a. Decrypt credentials
   b. Initialize cloud SDK client
   c. Call provider API
   d. Transform to ResourceInventory model
   e. Upsert to database
   f. Update provider_health
7. Task completes and fresh data becomes available to the API
```

K.4.9 Terraform Provisioning Flow

Source: `docs/SUMMARY.md`

Listing K.15: Terraform Provisioning Flow Adapted from docs/SUMMARY.md

```
1. User submits POST /resources/ with configuration
2. Create Resource record with status='pending'
3. Enqueue provision_resource_task(resource_id)
4. Celery worker picks up task
5. Generate Terraform .tf file from config
6. Run terraform init
7. Run terraform plan
8. Run terraform apply -auto-approve
9. Parse outputs (instance_id, public_ip, etc.)
10. Update Resource record:
    - status='active'
    - terraform_output={...}
11. Store Terraform state in terraform_states
12. User polls GET /resources/{id} for status
```

K.4.10 Repository Project Structure

Source: `docs/SUMMARY.md`

Listing K.16: Repository Project Structure Adapted from docs/SUMMARY.md

```
multi-cloud/
|-- backend/
|   |-- app/
|   |   |-- api/endpoints/
|   |   |-- core/
|   |   |-- db/
|   |   |-- models/
|   |   |-- schemas/
|   |   |-- services/
|   |   `-- tasks/
|   |-- main.py
|   |-- worker.py
|   `-- requirements.txt
|
|-- frontend/
|   |-- src/
|   |   |-- components/
|   |   |-- pages/
|   |   |-- services/
|   |   `-- App.tsx
|   `-- package.json
|
|-- docs/
|   |-- README.md
|   |-- BACKEND.md
|   |-- API.md
|   |-- DATABASE.md
|   |-- CELERY.md
|   |-- CLOUD_PROVIDERS.md
|   |-- SECURITY.md
|   `-- SUMMARY.md
|
|-- docker-compose.yml
|-- .env.example
`-- README.md
```

Bibliography

[1] Mell, Peter and Grance, Timothy. "The NIST Definition of Cloud Computing," *NIST Special Publication 800-145*, 2011.

[2] Brikman, Yevgeniy. *Terraform: Up & Running: Writing Infrastructure as Code*, O'Reilly Media, 2019.

[3] Armbrust, Michael, et al. "A view of cloud computing," *Communications of the ACM*, Vol. 53, No. 4, pp. 50-58, 2010.

[4] Buyya, Rajkumar, et al. "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Generation computer systems*, Vol. 25, No. 6, pp. 599-616, 2009.

[5] Morris, Kief. *Infrastructure as Code: Managing Servers in the Cloud*, O'Reilly Media, 2016.

[6] Al-Dhuraibi, Yahya, et al. "Elasticity in cloud computing: state of the art and research challenges," *IEEE Transactions on Services Computing*, Vol. 11, No. 2, pp. 430-447, 2018.

[7] Grozev, Nikolay and Buyya, Rajkumar. "Inter-Cloud architectures and application brokering: taxonomy and survey," *Software: Practice and Experience*, Vol. 44, No. 3, pp. 369-390, 2014.

[8] Zhang, Qi, et al. "Cloud computing: state-of-the-art and research challenges," *Journal of Internet Services and Applications*, Vol. 1, No. 1, pp. 7-18, 2010.

[9] Toosi, Adel Nadjaran, et al. "Interconnected cloud computing environments: Challenges, taxonomy, and survey," *ACM Computing Surveys (CSUR)*, Vol. 47, No. 1, pp. 1-47, 2014.

[10] Bernstein, David, et al. "Inter-cloud computing," *2009 IEEE International Conference on Cloud Computing*, IEEE, pp. 74-80, 2009.

[11] Polamarasetti, Shalini, et al. "Enhancing Strategic Business Decisions with AI-Powered Forecasting Models," *ASSIC 2025*, IEEE, 2025.

[12] HashiCorp. "Terraform Documentation and Architecture," 2024. [Online]. Available: https://developer.hashicorp.com/terraform

[13] Meta Open Source. "React Documentation," 2026. [Online]. Available: https://react.dev

[14] FastAPI Project. "FastAPI Documentation," 2026. [Online]. Available: https://fastapi.tiangolo.com

[15] Redis Ltd. "Redis Documentation," 2026. [Online]. Available: https://redis.io/docs/

[16] Docker Inc. "Docker Documentation," 2026. [Online]. Available: https://docs.docker.com/

[17] Amazon Web Services. "Boto3 Documentation," 2026. [Online]. Available: https://boto3.amazonaws.com

[18] Google Cloud. "Anthos Overview," 2024. [Online]. Available: https://cloud.google.com/anthos

[19] Beloglazov, Anton, et al. "Energy-aware resource allocation heuristics for efficient management of data centers," *Future Generation Computer Systems*, Vol. 28, No. 5, pp. 755-768, 2012.

[20] Calheiros, Rodrigo N, et al. "CloudSim: a toolkit for modeling and simulation of cloud computing environments," *Software: Practice and Experience*, Vol. 41, No. 1, pp. 23-50, 2011.

[21] Vaquero, Luis M, et al. "A break in the clouds: towards a cloud definition," *ACM SIGCOMM Computer Communication Review*, Vol. 39, No. 1, pp. 50-55, 2008.

[22] Singh, Sukhpal and Chana, Inderveer. "A Survey of Cloud Resource Management," *ACM Computing Surveys*, Vol. 48, No. 3, pp. 1-39, 2016.

[23] Celery Project. "Celery: Distributed Task Queue," 2024. [Online]. Available: https://docs.celeryq.dev

[24] HashiCorp. "Terraform Sentinel: Policy as Code for Infrastructure," 2024. [Online]. Available: https://www.hashicorp.com/products/terraform/sentinel

[25] Villari, Massimo, et al. "Leveraging the Intercloud," *IEEE Cloud Computing*, Vol. 3, No. 1, pp. 31-38, 2016.

[26] Rahman, Akond, et al. "The 5 W's of Infrastructure as Code: A Case Study," *2020 IEEE/ACM 42nd ICSE-SEIP*, IEEE, pp. 71-80, 2020.

[27] Bessani, Alysson, et al. "DepSky: Dependable and Secure Storage in a Cloud-of-Clouds," *ACM EuroSys Conference*, 2011.

[28] Villari, Massimo, et al. "Enabling Resource Discovery and Mapping in the Intercloud," *2013 IEEE SOSE*, IEEE, pp. 115-120, 2013.

[29] IEEE Computer Society, "IEEE Recommended Practice for Software Requirements Specifications," *IEEE Std 830-1998*, 1998.

[30] Pydantic Project, "Pydantic Documentation: Data validation and settings management using Python type hints," 2024. [Online]. Available: https://pydantic-docs.helpmanual.io